

Presents:

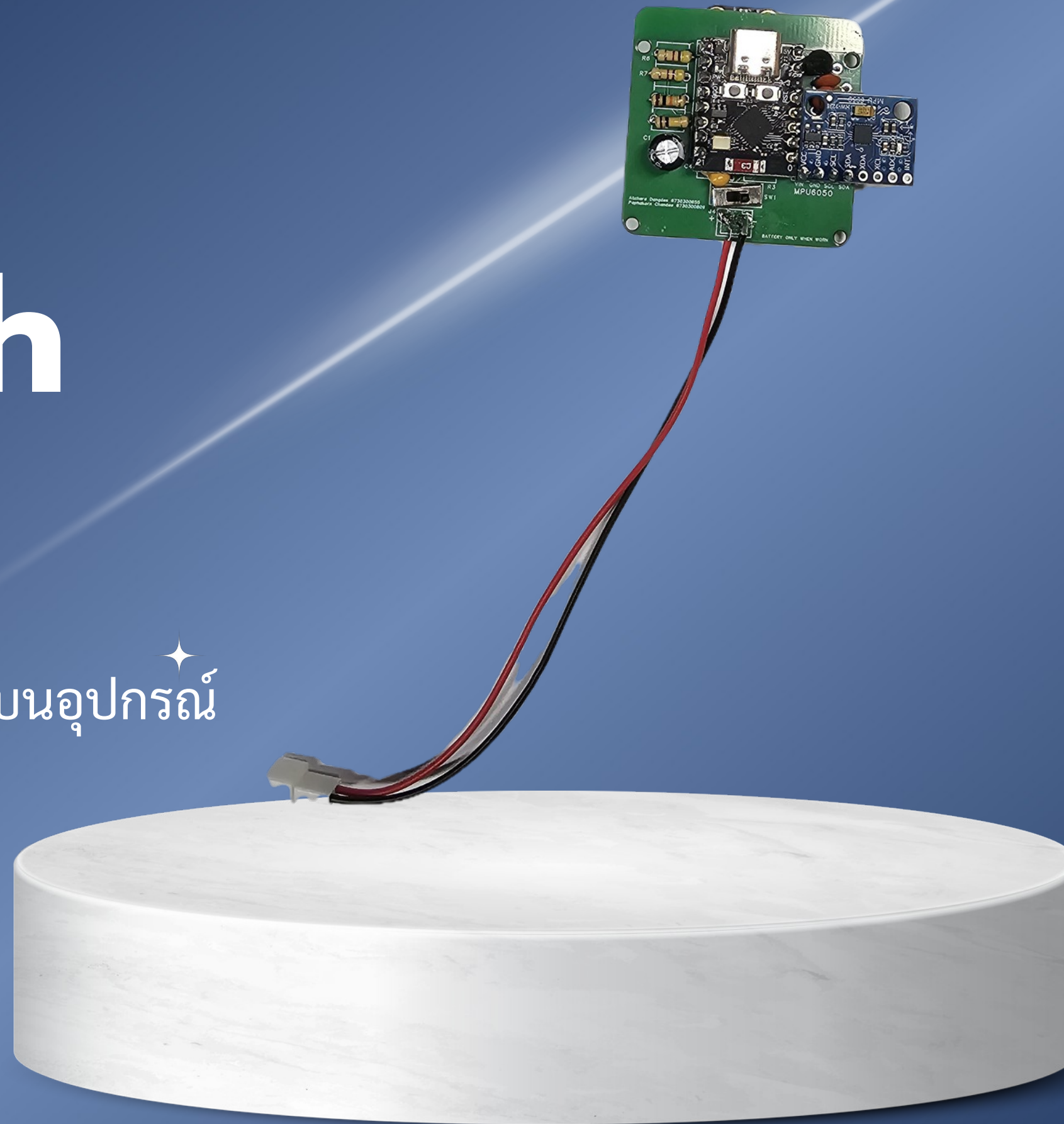
Wireless- Polygraph

2026

เครื่องจับเท็จไร้สายแบบสวมข้อมือ

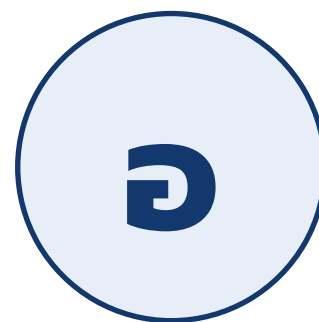
ESP32-C3 · FreeRTOS · WiFi · AI บนอุปกรณ์

03603323 Introduction to Embedded Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตศรีราชา



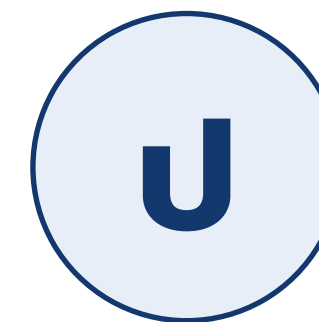
สมาชิก

Members



อัจฉรา ดังดี

6730300655



ปภากร จันทรดี

6730300809

วิศวกรรมคอมพิวเตอร์ มก. ศรีราชา · ภาคต้น ปีการศึกษา 2569

Agenda

01

Overview

ที่มา เป้าหมาย และภาพรวมระบบ

02

Hardware Design

Schematic, PCB, เซนเซอร์ 4 ตัว, แบตเตอรี่, ฐานยึด

03

Firmware & Embedded

FreeRTOS, I2C/ADC, หน่วยความจำ, Watchdog, Sleep

04

Software & UI

หน้าเว็บในนาฬิกา, Polygraph Studio บนคอม/มือถือ

05

AI & Backend

เก็บข้อมูลลง CSV, Label จริง/โกหก, เทรนและใช้บนนาฬิกา

06

Limitations & Solutions

ปัญหาที่เจอ สาเหตุ และวิธีแก้

07

Live Demo

สาธิตการใช้งานจริง

01

Overview

ที่มา เป้าหมาย และภาพรวมระบบ



ทำไมถึงอยากทำโครงการนี้

ปัญหา / แรงบันดาลใจ

เครื่องจับเท็จทั่วไป ใช้สายต่อหลายเส้น ตัวเครื่องใหญ่ ต้องนั่งติดกับคอม

ร่างกายฟ้องเมื่อตื่นเต้น ชีพจรเร่ง เหงื่อออก หลอดเลือดปลายมือหด มือสั่น ผิวย่นลง

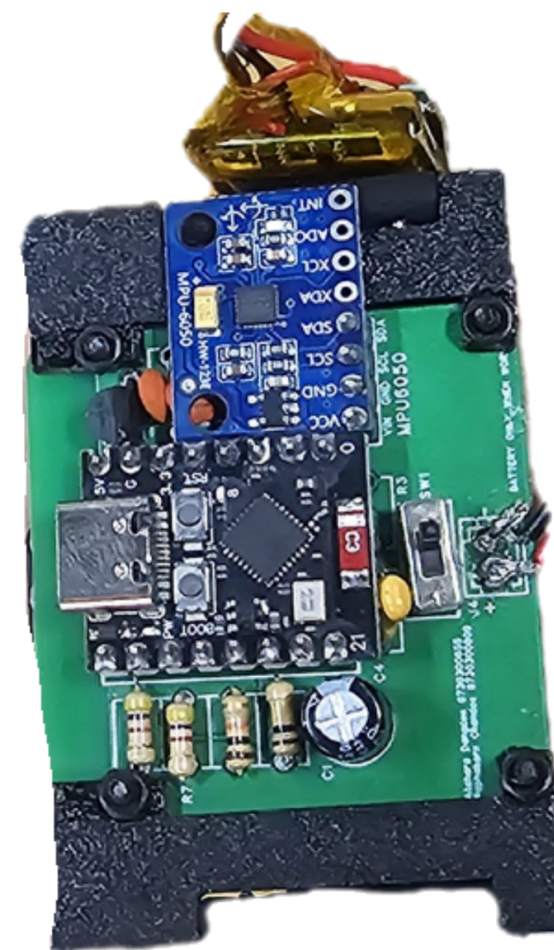
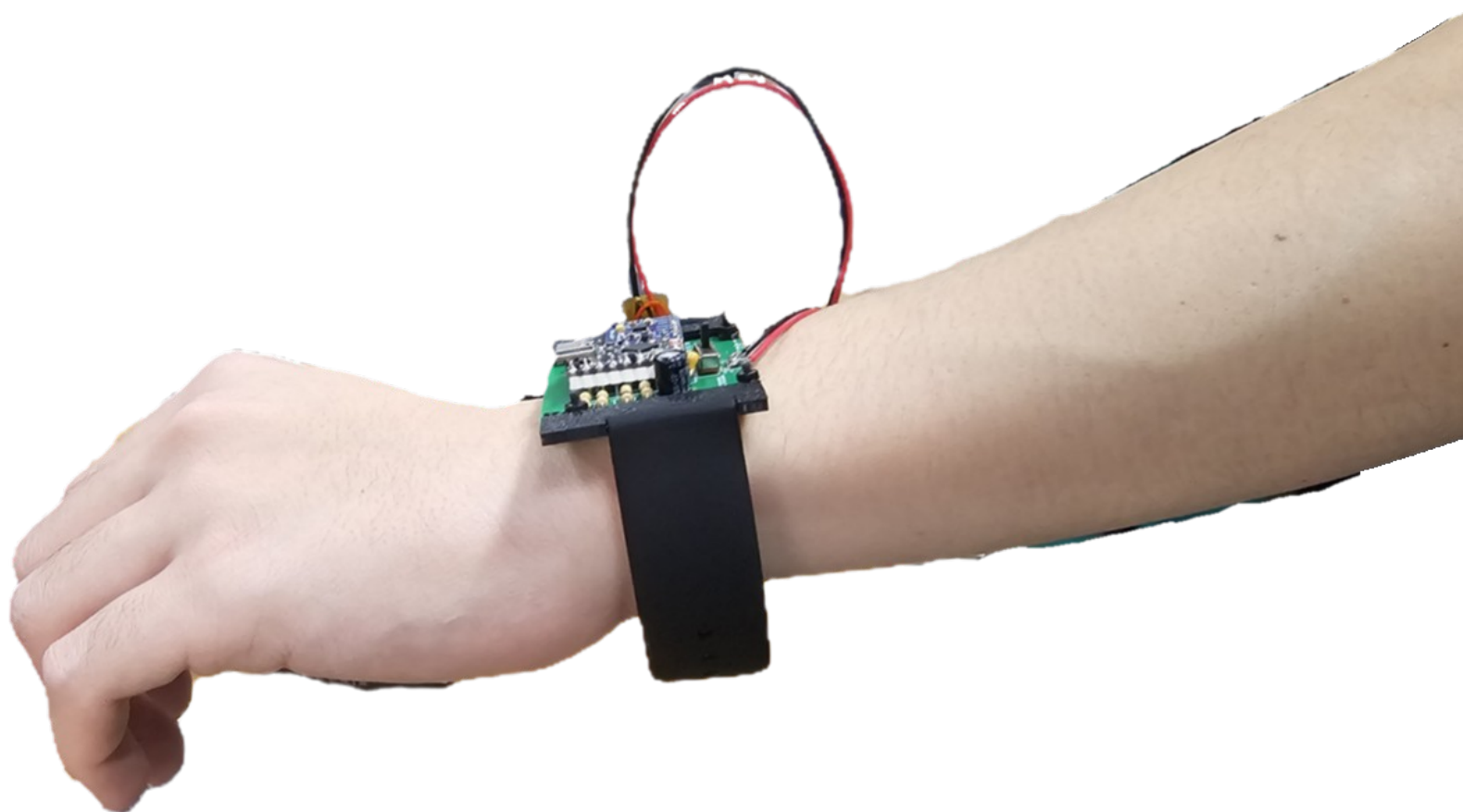
โจทย์ของเรา ทำให้เล็กพอใส่ข้อมือ ไร้สาย ดูผลได้จากมือถือ

โอกาสฝึกครบวงจร เซนเซอร์ → เฟิร์มแวร์ RTOS → WiFi → เว็บ → AI

เป้าหมาย

- 1 วัดสัญญาณร่างกาย 4 ชนิด ด้วยเซนเซอร์ที่ประกอบบน PCB ของเราเอง
- 2 ตัดสิน "จริง / โกหก" บนนาฬิกาเอง (edge computing) ไม่ต้องพึ่งคอม
- 3 ดูค่าสด กราฟ และผลได้ทั้งบนมือถือและคอมผ่าน WiFi
- 4 เก็บข้อมูลลง CSV เพื่อเทรน AI แล้วส่งโมเดลกลับเข้านาฬิกา

หน้าตาโดยรวม



4

เซนเซอร์สี่ระ

6

FreeRTOS task

5 Hz

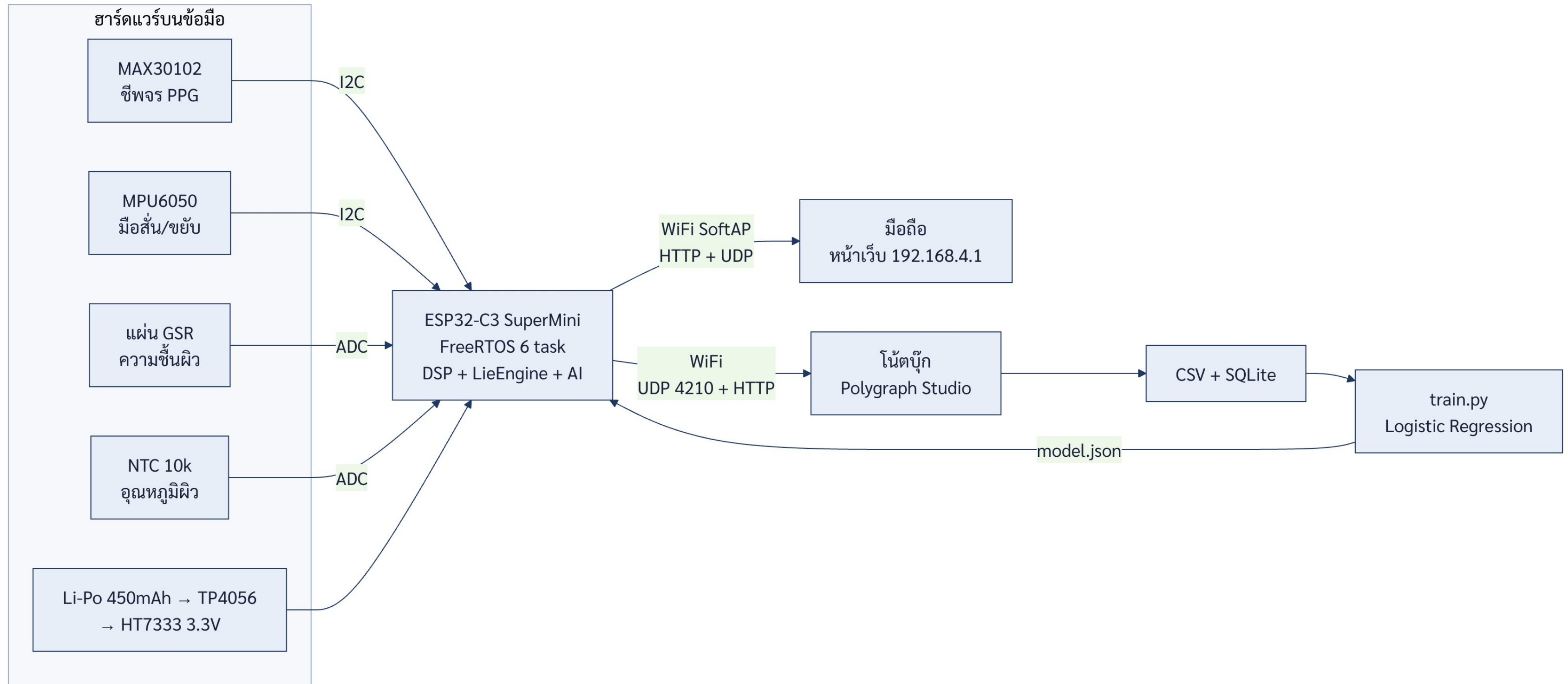
ส่งค่าสดผ่าน
WiFi

12

feature ของ AI

ESP32-C3 SuperMini สมองหลัก + WiFi ในตัว
Li-Po 3.7 V 450 mAh ชาร์จผ่าน TP4056 ใช้งานไร้สาย
ดูผลได้ 2 ทาง มือถือ (192.168.4.1) และโปรแกรมบนคอม

ภาพรวมระบบ



02

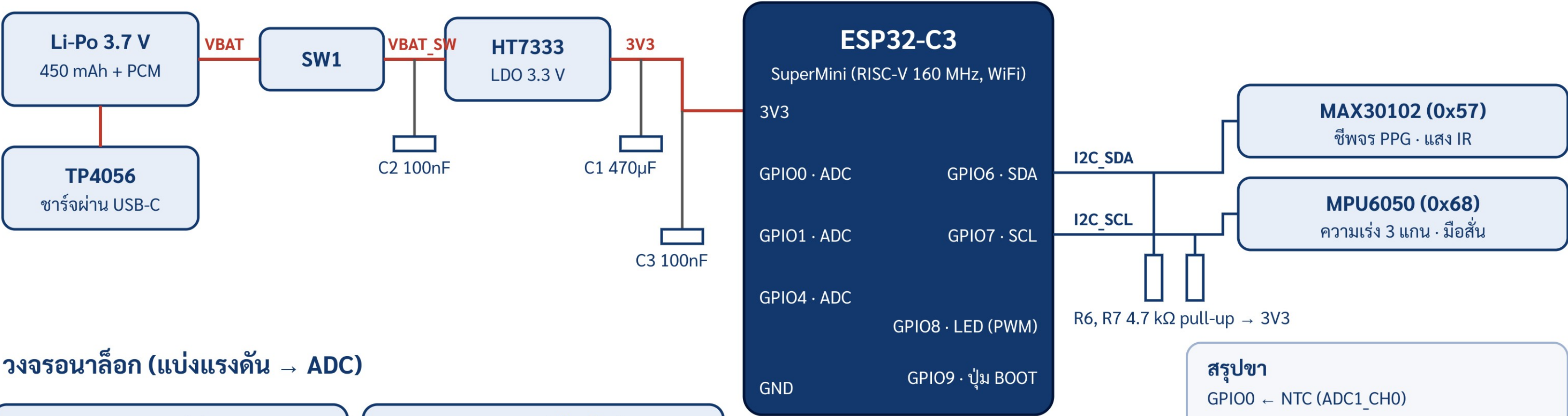
Hardware Design

Schematic · PCB · เซนเซอร์ · แหล่งจ่ายไฟ · ฐานยึด



Schematic และการต่อขา

แหล่งจ่ายไฟ



วงจรนาฬิกา (แบ่งแรงดัน → ADC)

อุณหภูมิผิว

3V3 — NTC 10k — R1 10k — GND

L → NTC_SENSE (GPIO0)

สมการ Beta (B = 3950) → °C

GSR (ความนำไฟฟ้าผิว)

3V3 — R2 100k — แผ่น TP1 ~ ผิว ~ TP2

TP2 — R3 100k — GND (+C4 1µF กรอง)

L → GSR_SENSE (GPIO1)

$G = 1 / R \text{ ผิว} \rightarrow \mu\text{S}$

วัดแบตเตอรี่

VBAT — R4 100k — R5 100k — GND

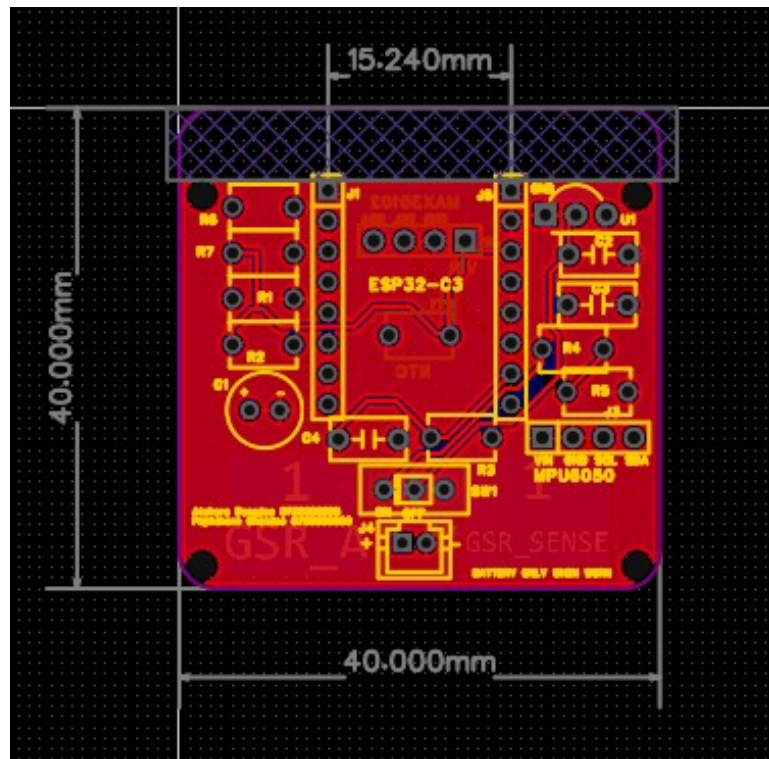
L → VBAT_SENSE (GPIO4) ×2

สรุปขา

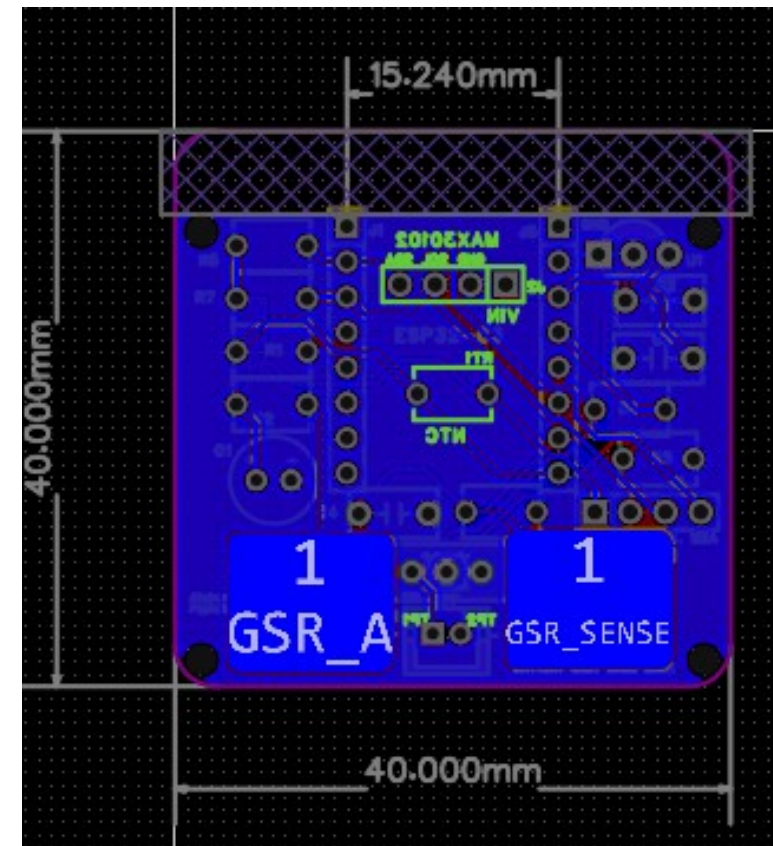
- GPIO0 ← NTC (ADC1_CH0)
- GPIO1 ← GSR (ADC1_CH1)
- GPIO4 ← แบต ÷2 (ADC1_CH4)
- GPIO6/7 ↔ I2C 400 kHz
- GPIO8 → LED (LEDC PWM)
- GPIO9 ← ปุ่ม BOOT
- PCB 40 × 40 mm, 2 ชั้น

ออกแบบด้วย EasyEDA · ใช้ ADC1 ทั้งหมด (ADC2 ใช้งานไม่ได้ขณะเปิด WiFi) · I2C pull-up 4.7 kΩ

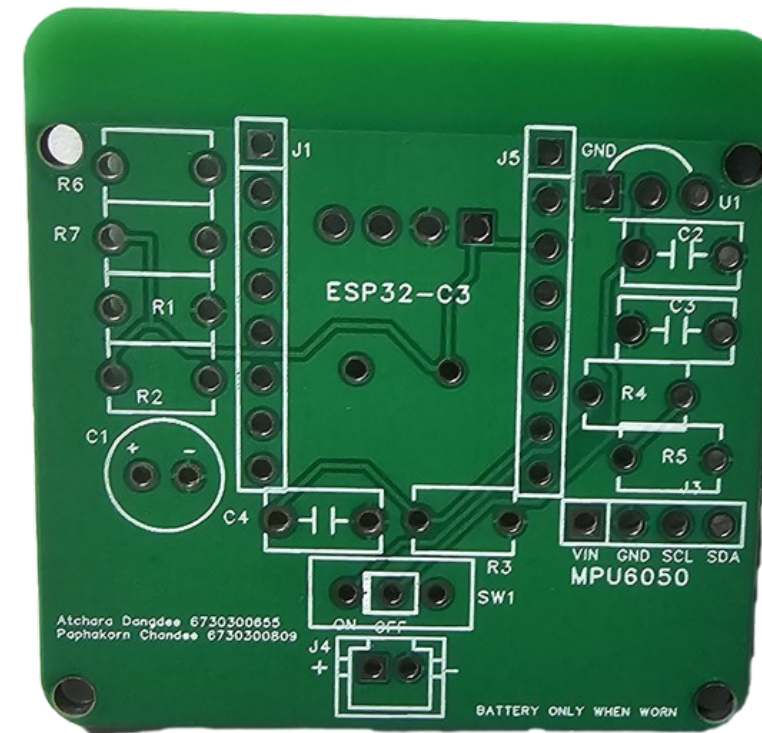
PCB



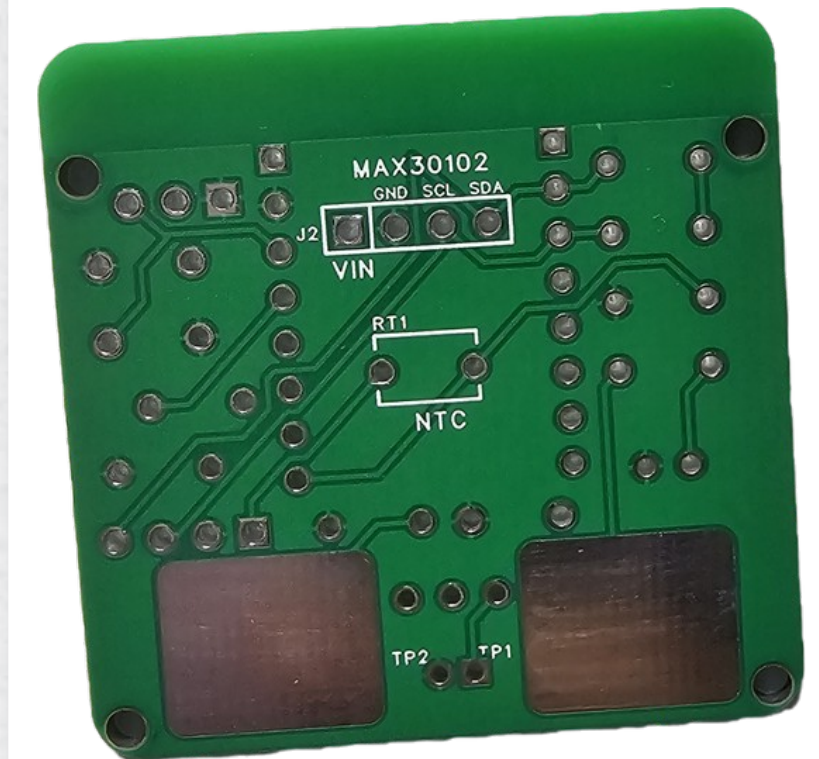
ด้านบน (Top layer)



ด้านล่าง (Bottom layer)



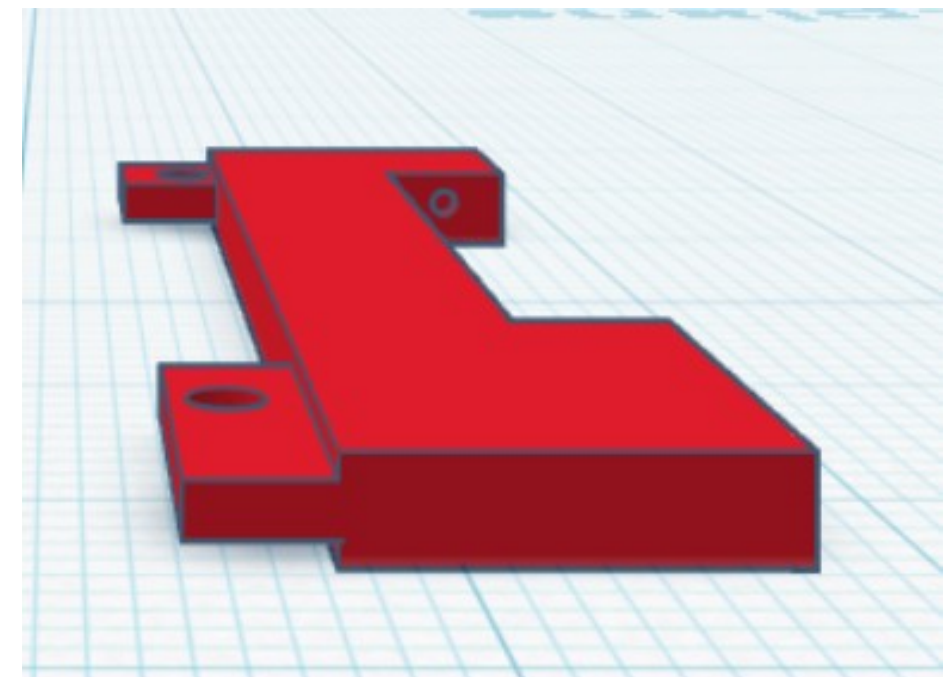
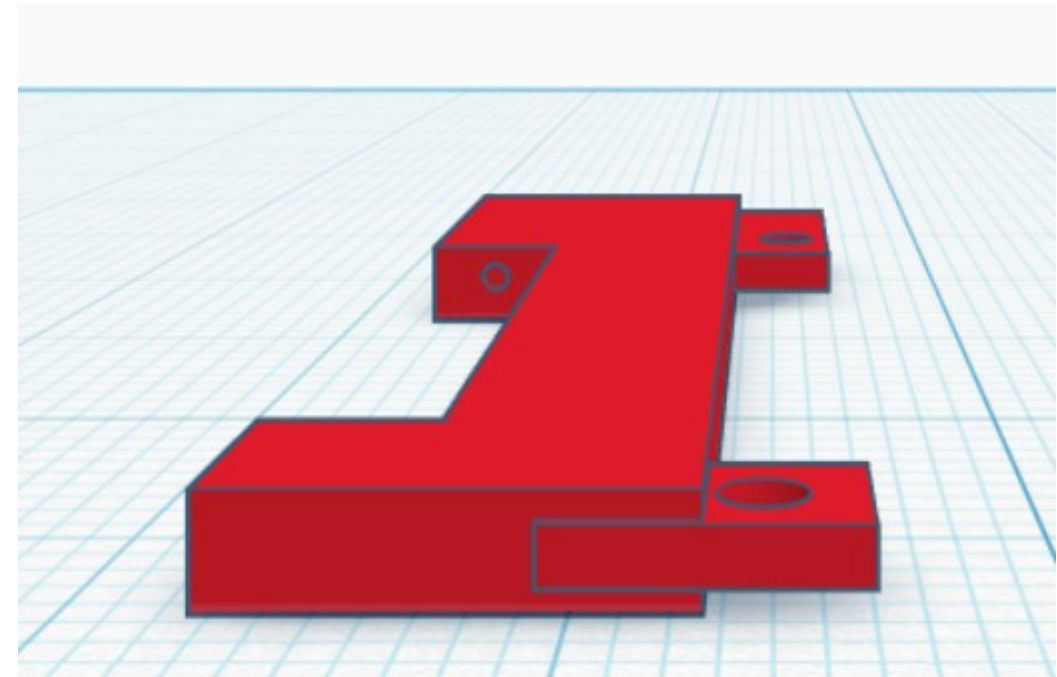
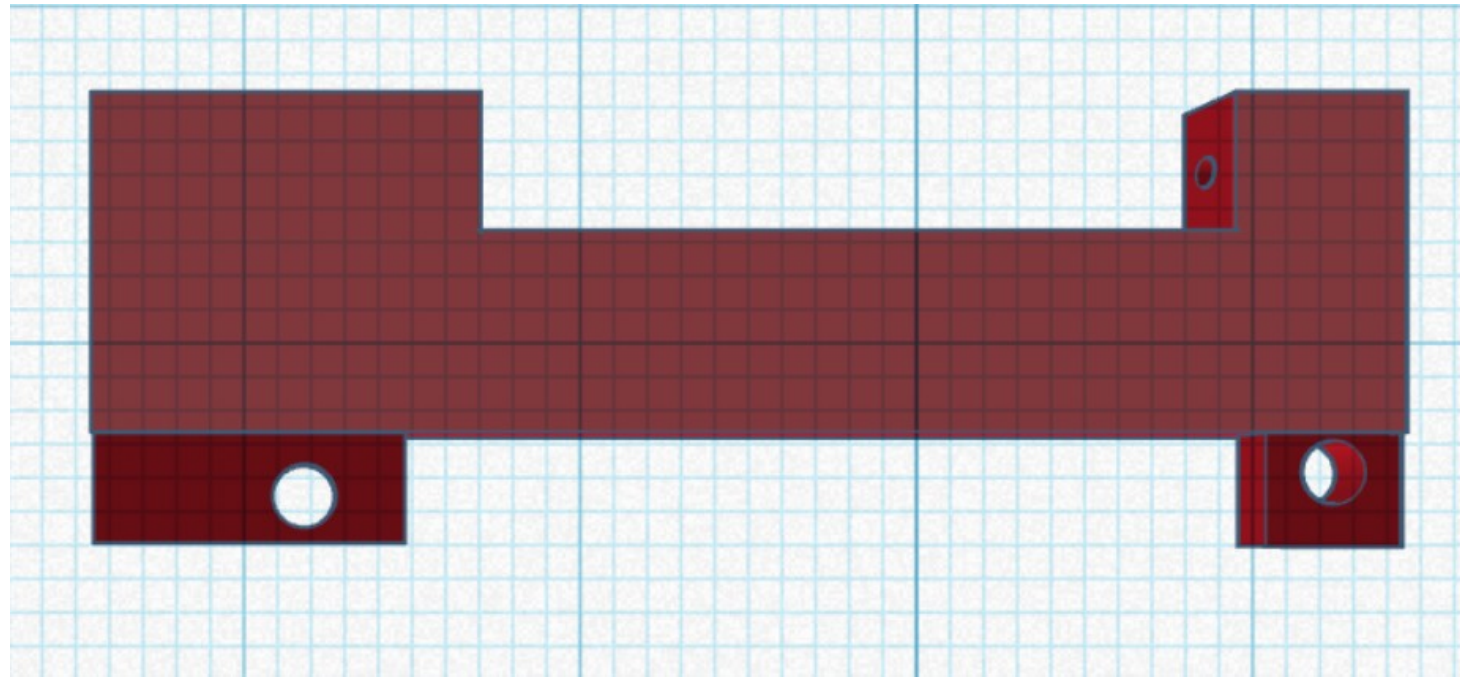
ภาพจริง ด้านบน

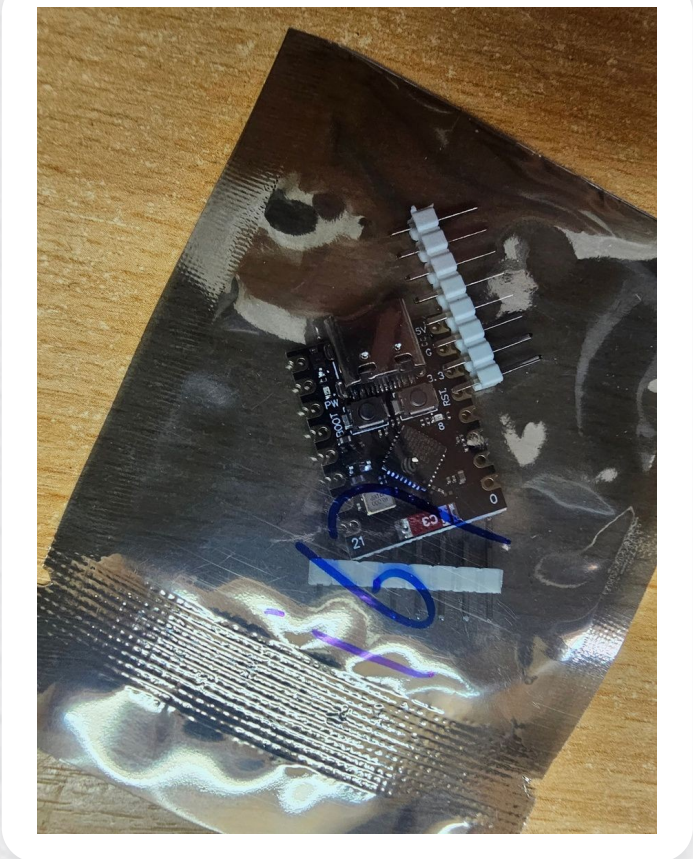


ภาพจริง ด้านล่าง — แผ่น GSR

2 ชั้น • 40 × 40 มม. วางบอร์ด ESP32-C3 บนซ็อกเก็ต ถอดเปลี่ยนได้
แผ่น GSR บน PCB ใช้ลายทองแดงเป็นขั้วสัมผัสผิว ไม่ต้องมีสายแยก

ฐานยึดสายและตัวบอร์ด





- RISC-V 32-bit 160 MHz
- SRAM 400 KB
- RTC memory 8 KB
- Flash 4 MB
- WiFi 2.4 GHz + BLE 5

GPIO6/7 I2C (MAX30102, MPU6050)
GPIO0/1/4 ADC (NTC, GSR, แบต)
GPIO8/9 LED PWM / ปุ่ม BOOT

เซนเซอร์ 4 ชนิด



MAX30102

ชีพจร (PPG)

แสง IR สะท้อนจากเส้นเลือด →
ชีพจร, HRV, แรงชีพจร

I2C 0x57



MPU6050

การสั่น / ขยับ

ความเร่ง 3 แกน → มีอัส 3-
15 Hz, การขยับตัว

I2C 0x68

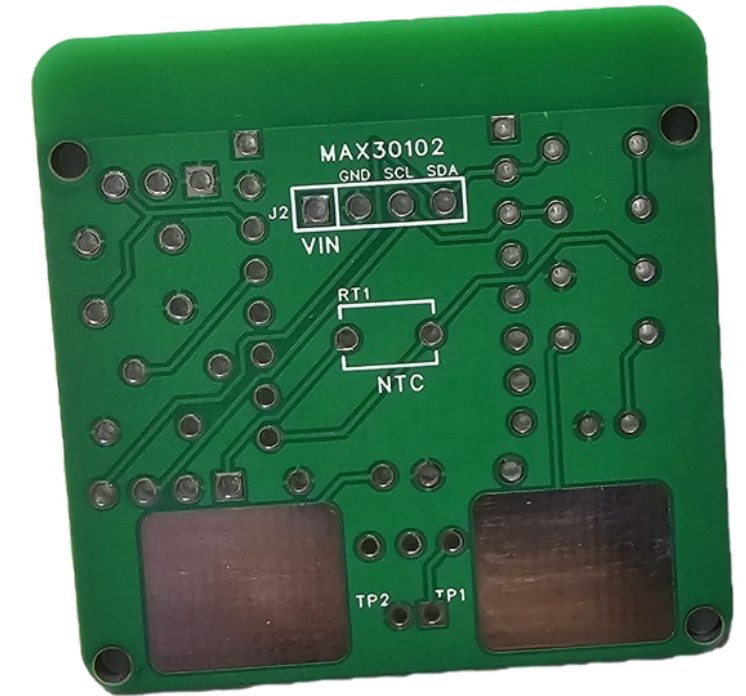


NTC 10 k Ω

อุณหภูมิผิว

ความต้านทานเปลี่ยนตาม
อุณหภูมิ → °C

ADC GPIO0



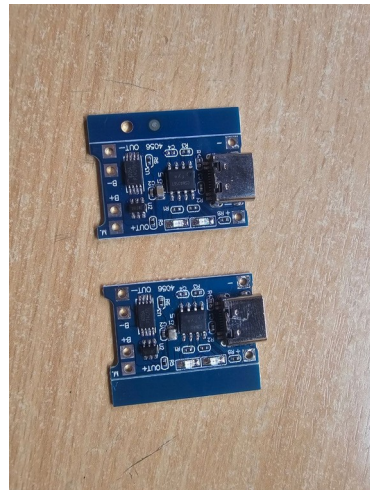
แผ่น GSR

ความชื้นผิว

เหงื่อทำให้ผิวนำไฟฟ้าดีขึ้น →
 μS

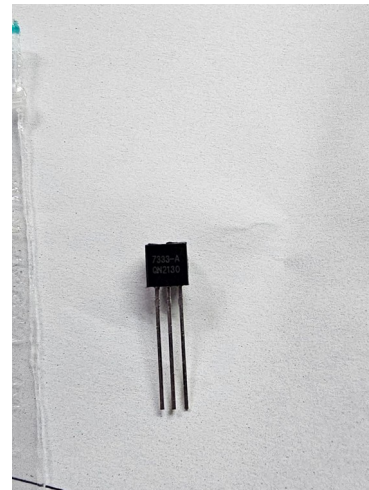
ADC GPIO1

อุปกรณ์อื่น ๆ



TP4056

ชาร์จแบตเตอรี่ Li-Po
ผ่าน USB-C



HT7333

LDO แปลงแบตเตอรี่
→ 3.3 V



สวิตช์เลื่อน

เปิด/ปิดไฟจาก
แบตเตอรี่



ตัวเก็บประจุ เซรามิก

decoupling ไกลล์
ขาชิป



R 4.7 k Ω

I2C pull-up



R 10 k Ω

คู่กับ NTC



R 100 k Ω

วงจร GSR + วัด
แบตเตอรี่



ตัวเก็บประจุ MLCC

กรองสัญญาณ
GSR และไฟเลี้ยง

ระยะเวลาใช้งานแบตเตอรี่



ทางทฤษฎี (ประมาณการ)

ส่วนประกอบ	กระแสเฉลี่ย
ESP32-C3 + WiFi SoftAP	≈ 90 mA *
MAX30102 (LED IR)	≈ 2 mA *
MPU6050	≈ 4 mA *
NTC/GSR/วัดแบตเตอรี่	< 0.2 mA
รวม	≈ 100 mA

$450 \text{ mAh} \times 80\% \text{ (ตัดที่ } 3.3 \text{ V)} \div 100 \text{ mA}$

≈ 3.6 ชั่วโมง

** ประมาณจาก datasheet และการตั้งค่าของเรา (LED IR ≈ 6.2 mA แบบพัลส์, WiFi TX 8.5 dBm) ไม่ใช่ค่าที่วัดได้*

ทางปฏิบัติ: วิธีวัดจริง

- ชาร์จเต็ม 4.2 V แล้วถอด USB
- เปิด WiFi + ส่งข้อมูลสดต่อเนื่อง (เหมือนใช้งานจริง)
- บันทึก battery_mv ทุก 0.2 วินาทีลง CSV
- จับเวลาจนแบตเตอรี่ลดถึง 3.3 V (นาฬิกาเข้า deep sleep ป้องกันแบตเตอรี่เอง)
- ลดการกินไฟ: ECO 80 MHz, ลดกำลังส่ง WiFi, หลับอัตโนมัติ

ระยะเวลาการทำงาน



ปี พ.ศ. 2569 · ช่วงก่อนสอบกลางภาคเน้นฮาร์ดแวร์ หลังสอบกลางภาคเน้นเฟิร์มแวร์ ซอฟต์แวร์ และ AI

03

Firmware & Embedded

FreeRTOS · Interrupt · I2C · ADC · Memory · Watchdog ·
Sleep



เทคนิค Embedded ที่ใช้ในโครงการ

1 GPIO + PWM

LED บอกสถานะ (LEDC 5 kHz)
· ปุ่ม BOOT + debounce

2 Interrupt + Timer

Hardware timer 100 Hz ปลุก
sensorTask จาก ISR

3 UART / USB Serial

Serial console พิมพ์คำสั่ง
status · i2c · mode

4 I2C

MAX30102 + MPU6050 ที่
400 kHz · ภูบัสค่างอัตโนมัติ

5 ADC

GSR · NTC · แบต — ADC1,
เฉลี่ย 16 ครั้ง

6 Memory

Flash map 4 MB · NVS ·
EEPROM · RTC · LittleFS

7 WiFi

SoftAP + UDP ค่าสด 5 Hz +
HTTP REST API

8 Watchdog

3 ชั้น + กล้องดำบอกสาเหตุ
รีเซ็ต

9 Low power

ECO 80 MHz · Light / Deep
sleep · Standby

10 RTOS

6 task · queue · mutex ·
event group

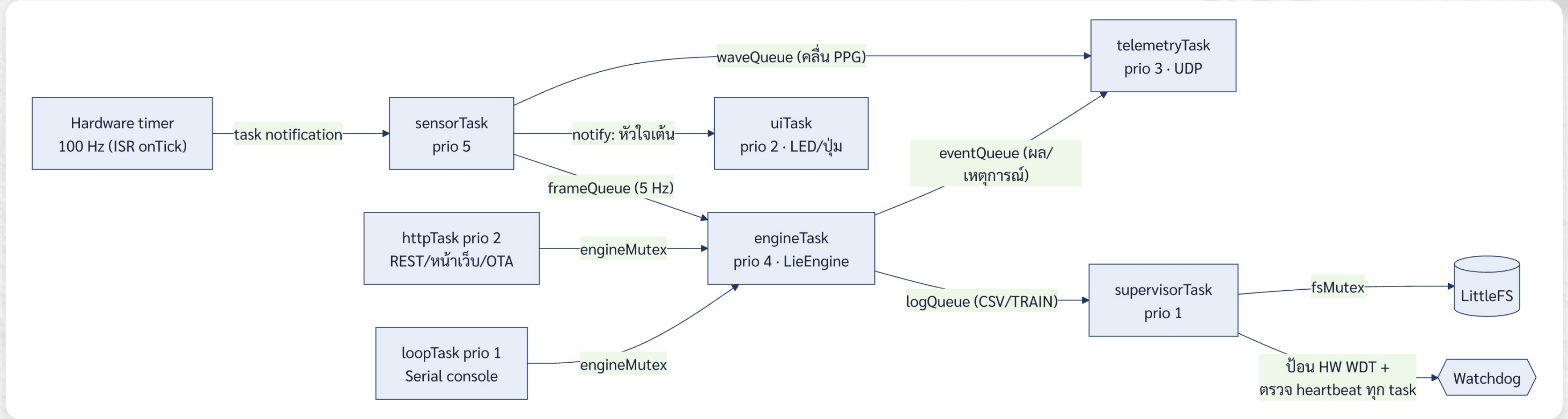
11 PCB + 3D

PCB 2 ชั้น 40×40 มม. · ฐานยึด
3 มิติ

12 Intelligence

Logistic Regression คำนวณ
บนนาฬิกาเอง

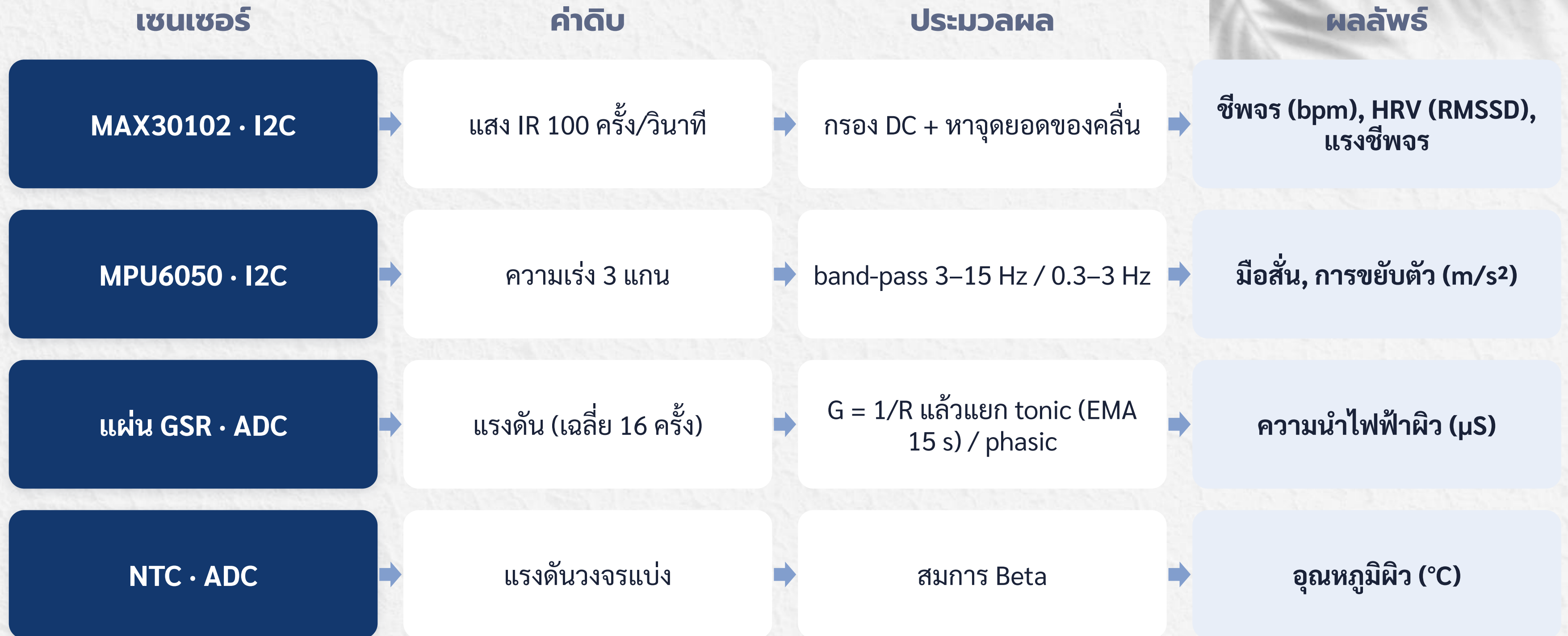
FreeRTOS: แบ่งงานเป็น 6 task



task	priority	หน้าที่
sensorTask	5	อ่านเซนเซอร์ทุก 10 ms (ถูกปลุกจาก timer interrupt)
engineTask	4	LieEngine + โมเดล AI ตัดสินจริง/โกหก 5 ครั้ง/วินาที
telemetryTask	3	ส่งค่าสด/เหตุการณ์ทาง UDP

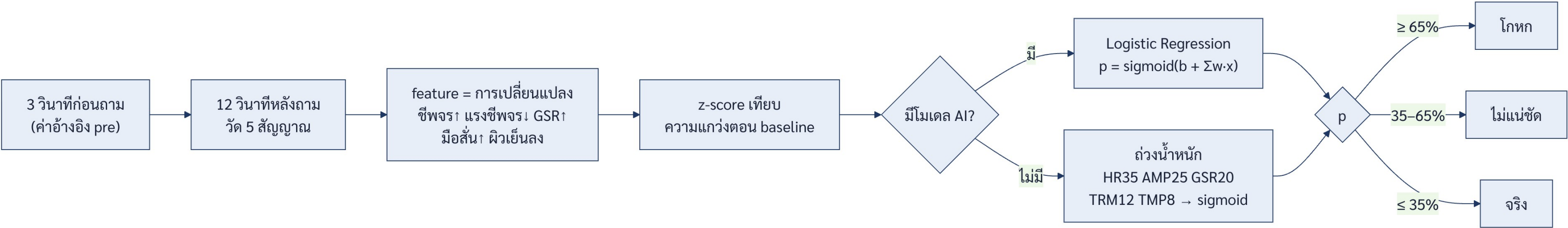
task	priority	หน้าที่
httpTask	2	หน้าเว็บ + REST API + OTA
uiTask	2	LED กระพริบตามหัวใจ + ปุ่ม
supervisorTask	1	เขียนแฟลช · เช็คแบตเตอรี่ · ป้อน watchdog

อ่านเซนเซอร์และประมวลผลสัญญาณ

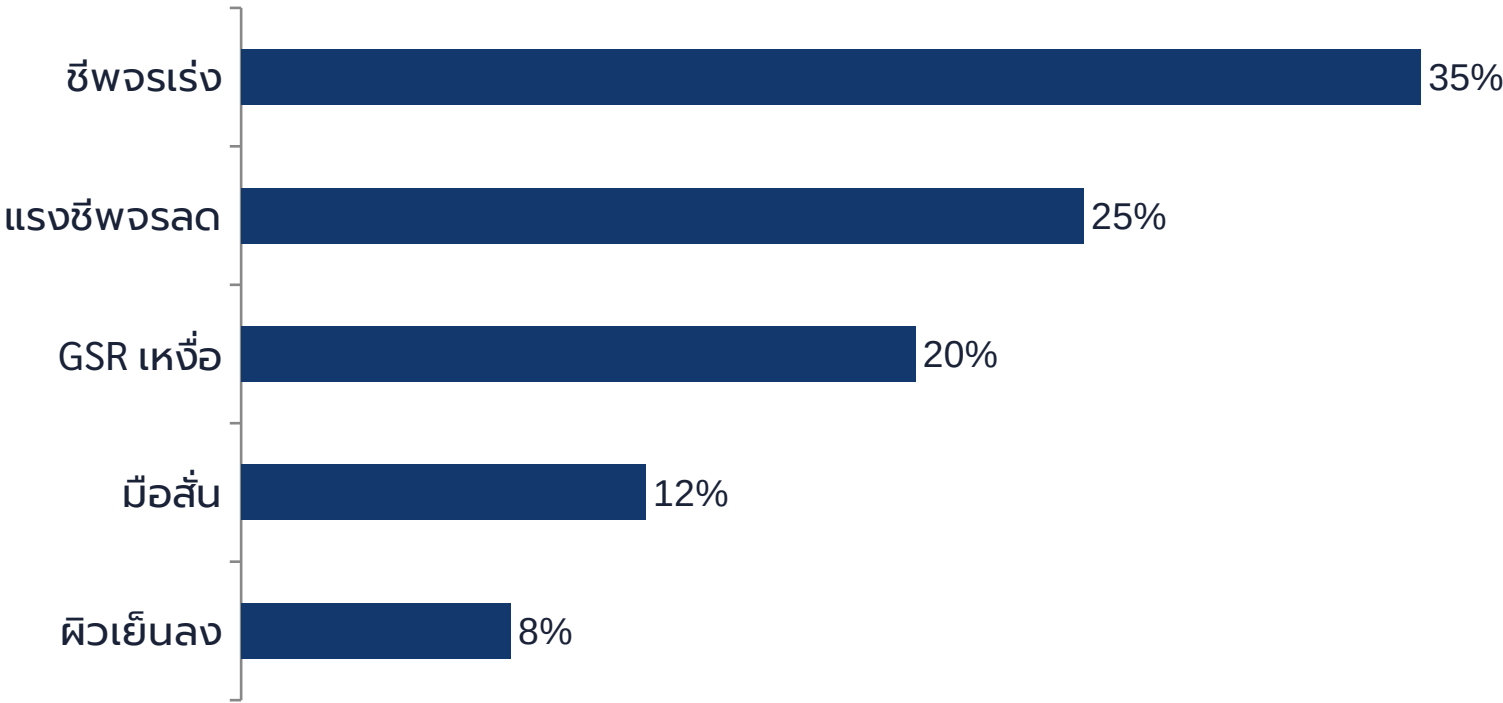


รวมเป็น frame ทุก 200 ms → frameQueue → engineTask (LieEngine) และส่งทาง UDP ให้กราฟเรียลไทม์

ตัดสินใจจริง / โททกอย่างไร



น้ำหนักเริ่มต้นของแต่ละสัญญาณ



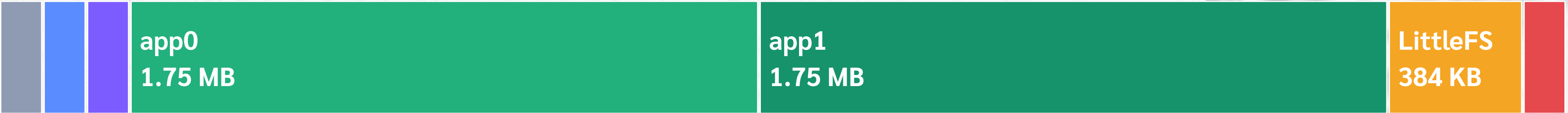
Baseline นั่งพัก 30 วินาที เพื่อรู้ "ค่าปกติ" และความกว้างของคนนั้น

z-score เปลี่ยนมากกว่าตอนพักก็เท่า — ทำให้เทียบคนต่างกันได้

ข้อควบคุม ข้อที่รู้เฉลย ใช้ปรับน้ำหนัก/เกณฑ์ให้เข้ากับแต่ละคน

ไม่มีเหงื่อก็ได้ GSR หาย → ใช้ 4 สัญญาณที่เหลือแทน

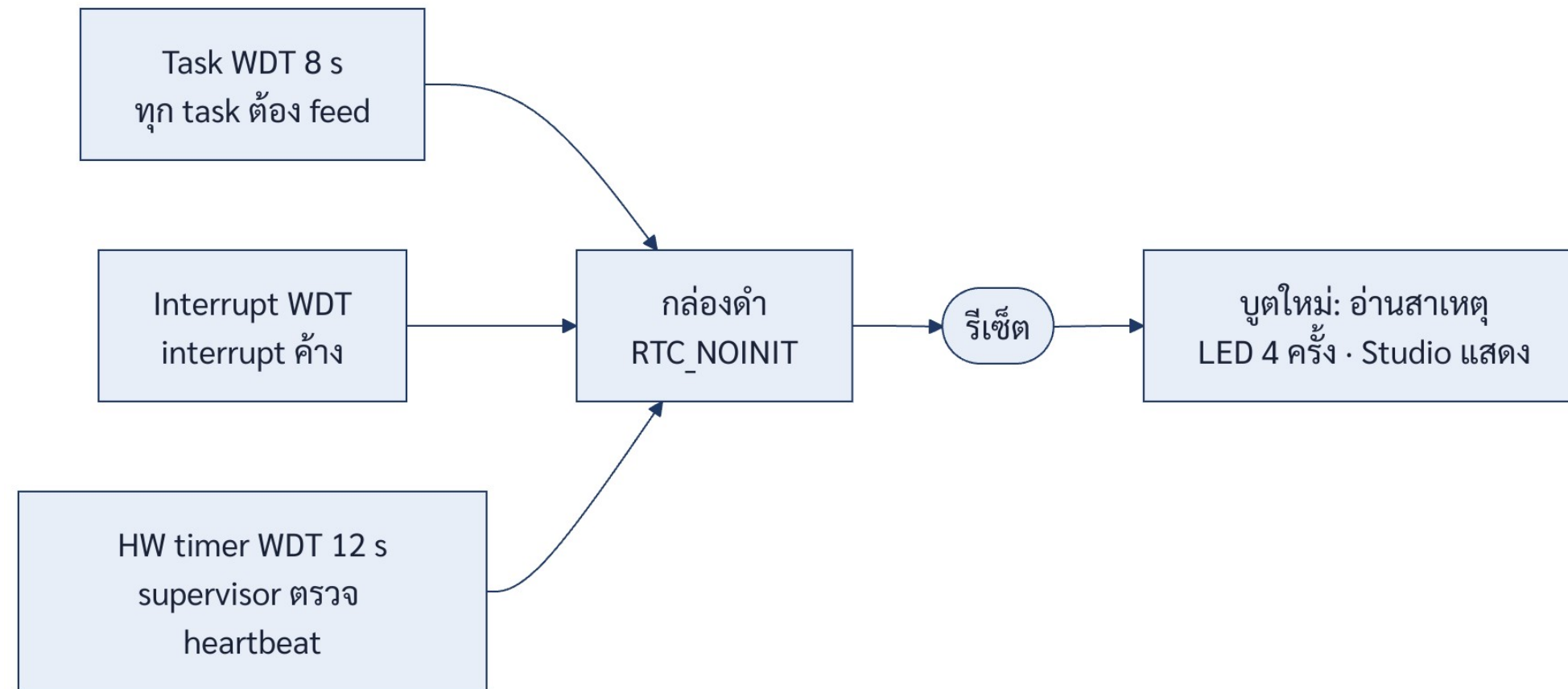
หน่วยความจำ: Flash Map 4 MB



0x0000 bootloader · 0x9000 nvs · 0xE000 otadata · 0x10000 app0 · 0x1D0000 app1 · 0x390000 LittleFS · 0x3F0000 coredump

ที่เก็บ	อยู่ที่	เก็บอะไร	ทำอะไรได้
NVS (Preferences)	Flash	ค่าตั้ง, โหมด, โมเดล AI (+ CRC32)	ไฟดับ · wear leveling ในตัว
EEPROM emulation	Flash (NVS blob)	สถิติสะสม: บุต, watchdog, ไฟตก	ไฟดับ (ต้อง commit)
LittleFS	Flash 384 KB	events.log, results.csv, train.csv	ไฟดับ · ดาวน์โหลดผ่านเว็บได้
RTC_DATA_ATTR	RTC memory 8 KB	ตัวนับการตื่นจาก deep sleep	deep sleep
RTC_NOINIT_ATTR	RTC memory	"กล่องดำ" เหตุผลการรีเซ็ต	WDT · panic · restart
SRAM (stack/heap)	SRAM 400 KB	ตัวแปร, คิว, stack ของ 6 task	หายเมื่อรีเซ็ต

Watchdog 3 ชั้น + กล้องดำ



Task WDT · 8 วินาที

1

ทุก task ต้องเรียก feed() — task ไหนค้างรู้ทันที

Interrupt WDT

2

interrupt ค้าง/ปิด interrupt นานเกิน → รีเซ็ต

Hardware timer WDT · 12 วินาที

3

supervisor ตรวจ heartbeat ทุก task แล้วค่อยบ๊อง

หลังรีเซ็ตเกิดอะไรขึ้น

- อ่านกล้องดำใน RTC_NOINIT + core dump → รู้สาเหตุและชื่อ task ที่ค้าง
- ไฟ LED กระพริบ 4 ครั้งตอนบูต, Studio แจ้งเตือน, คำสั่ง boots แสดงประวัติ 8 ครั้งล่าสุด
- สาธิตได้จากหน้า "ระบบ & อุปกรณ์" หรือ Serial: wdt twdt confirm / hwwdt / panic / intwtdt

พลังงานและ Sleep Mode

โหมด	CPU	WiFi	หน่วยความจำ	ปลุกด้วย	ใช้เมื่อ
Normal	160 MHz	เปิด	อยู่ครบ	-	ใช้งานปกติ
ECO	80 MHz	เปิด	อยู่ครบ	-	ประหยัดไฟ ส่งข้อมูลช้าลง
Light sleep	หยุด	ปิด	อยู่ครบ (baseline ไม่หาย)	ปุ่ม BOOT / timer	พักชั่วคราว
Deep sleep	ปิด	ปิด	เหลือแค่ RTC memory	timer → บุตใหม่	เลิกใช้ / แบตต่ำ < 3.3 V
Standby	ปิด	ปิด	RTC memory	ตื่นเช็คทุก 20 วินาที	รอคนมาใส่نافิกา

ตั้งค่าง่าย ๆ

หน้าเว็บ เลือก "หลับอัตโนมัติ" ปิด / 5 / 10 / 30 / 60 นาที
Serial sleep off · sleep auto 10 · sleep light · sleep deep 60
ปุ่ม **BOOT** กดค้าง 5 วินาที = light sleep, 10 วินาที = standby

ไฟ LED บอกสถานะ (ไม่ต้องมีจอ)

- กระพริบ 2 / 4 / 6 ครั้งตอนบุต = ปกติ / watchdog / ไฟตก
- ติดค้าง 0.6 วินาที = เปิด WiFi สำเร็จ
- กระพริบตามจังหวะหัวใจ = วัดชีพจรอยู่
- รัว 3 วินาที = ผลโกหก · ติดค้าง = ผลจริง

04

Software & UI

หน้าเว็บในนาฬิกา · Polygraph Studio · มือถือ /
คอมพิวเตอร์



Frontend / Backend 2 ชั้น

ชั้นที่ 1 — ในนาฬิกา

Backend REST API ใน ESP32 (web_server.cpp)

Frontend หน้าเว็บ HTML/JS ฝังในแฟลช

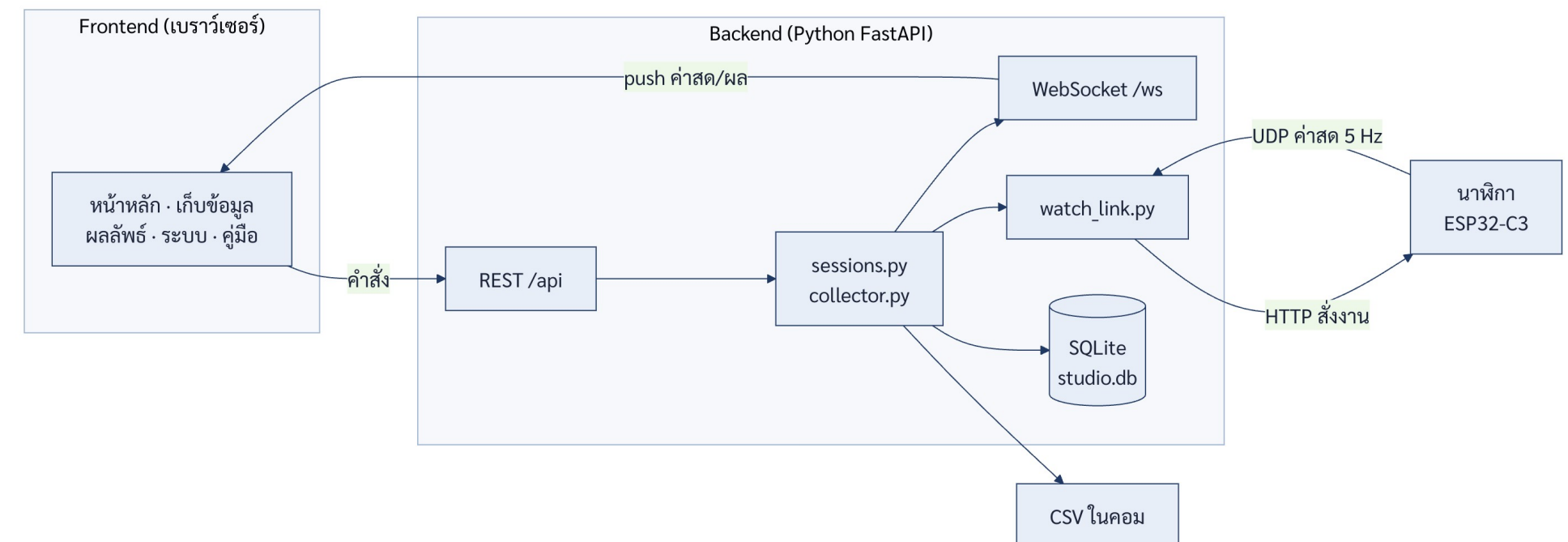
เปิดที่ <http://192.168.4.1> บนมือถือ

ชั้นที่ 2 — บนคอม (Studio)

Backend Python FastAPI + SQLite + WebSocket

Frontend หน้าเว็บ 5 หน้า + กราฟเรียลไทม์

เปิดที่ `run_studio.bat` → `127.0.0.1:8000`



Polygraph Studio มี 5 หน้า ทำอะไรบ้าง

1

หน้าแรก

ใช้งานจริง

ถามคำถามจริง นาฬิกาตัดสินใจ
โกหก/จริง แล้วตอบว่าถูก
หรือผิด เลือกได้ว่าตัดสินด้วย
โมเดล AI หรือสูตรมาตรฐาน

ได้อะไรออกมา

ผลรายข้อ + ความแม่นยำตอนใช้
งานจริง (result_*.csv, mode =
live)

2

เก็บข้อมูลเทรน AI

ถามข้อที่รู้เฉลย 2 แบบ: fix
(บอกก่อนถาม) / manual (ผู้
ตอบบอกทีหลัง) นับถอยหลัง
12 วินาทีทุกข้อ

ได้อะไรออกมา

ข้อมูลเทรน result_*.csv + ค่าสด
signals_*.csv

3

ข้อมูล & เทรน AI

เลือกไฟล์ที่ใช้เทรน
นำเข้า/รวม/ลบไฟล์ กดเทรน
AI แล้วส่งโมเดลเข้านาฬิกา
(อัตโนมัติ)

ได้อะไรออกมา

model.json ในคอม → เข้านาฬิกา

4

ระบบ & อุปกรณ์

ดูหน่วยความจำ FreeRTOS
task watchdog พลังงาน
สาธิต watchdog ตั้งค่า
LieEngine อัปเดตเฟิร์มแวร์ OTA
ดู log

ได้อะไรออกมา

ค่าตั้งใน NVS ของนาฬิกา,
firmware ใหม่, ไฟล์ log

5

คู่มือ & หลักการ

สรุปขั้นตอนใช้งาน หลักการ
ตัดสินโกหก/จริง ปุ่มบน
นาฬิกา และลิงก์ Data
Dictionary

ได้อะไรออกมา

ใช้อ่านอย่างเดียว

- **แถบบนทุกหน้า** สถานะเชื่อมต่อนาฬิกา (จริง/จำลอง) · สถานะเครื่องตัดสิน (ว่าง / วัด baseline / พร้อมถาม / กำลังวัด) · แบต % และแรงดัน · ความหน่วงและแพ็กเก็ต UDP ที่หาย · ปุ่มสลับโหมดสว่าง/มืด
- **เมนูซ้าย** "หน้าเว็บในนาฬิกา ↗" เปิด 192.168.4.1 (หน้าเว็บที่ฝังในเฟิร์มแวร์) · การ์ดสถานะนาฬิกา (รุ่นเฟิร์มแวร์, ครั้งที่บูต, IP) · ผู้จัดทำ

หน้าใช้งานจริง (หน้าแรก)

© ถามคำถาม

ได้ผลแล้ว — กดบอกว่านาฬิกาตอบถูกหรือผิด

© วัดค่าปกติ (baseline)

คำถาม (พิมพ์ไว้เพื่อบันทึก ไม่บังคับ)

คุณกินข้าวเช้าแล้วใช่ไหม?

ถาม (บันทึก 12 วินาที) ยกเลิกข้อนี้

ข้อที่ 3 - คุณกินข้าวเช้าแล้วใช่ไหม?

พูดจริง

โอกาสโกหก 3% · ตัดสินด้วยสูตรมาตรฐาน · คุณภาพสัญญาณ 100%

นาฬิกาตอบถูกไหม?

ถูก ผิด ไม่ทราบ

จบการใช้งาน data/result_20261009_155437.csv

วิธีตัดสินของนาฬิกา

ตัดสินด้วย โมเดล AI สูตรมาตรฐาน

โมเดลล่าสุดแม่นยำ 50% ไม่สูงกว่าการเดา "คำตอบที่พบบ่อยที่สุด" ทุกข้อ (54%) — แนะนำใช้ สูตรมาตรฐาน จนกว่าจะเก็บข้อมูลเพิ่มแล้วเทรนใหม่

ตอนนี้ นาฬิกาใช้ สูตรมาตรฐาน

โมเดลในนาฬิกา lr-20261009-125530 (แม่นยำตอนเทรน 50%, 209 ข้อ)

ล่าสุดในคอม lr-20261009-125530 - 50%

ส่งอัตโนมัติ เปิด ตรงกัน

ตัดสินด้วยสูตรมาตรฐาน (โมเดลยังเก็บไว้ในนาฬิกา) · ตั้งค่าโมเดลที่หน้า ข้อมูล & เทรน AI

รอบนี้

ถูก ผิด ไม่ทราบ ความแม่นยำ

2 0 0 100%

ข้อ	คำถาม	นาฬิกาทาย	P	ถูก/ผิด
2	คุณชอบวิชา Embedded ใช่ไหม?	พูดจริง	4%	ถูก
1	เมื่อวานคุณไปเรียนใช่ไหม?	โกหก	89%	ถูก

ลบข้อที่ถามผิด/ทดลอง: ดึงหรือกด ป ในตาราง "ประวัติการใช้งานจริง" ด้านล่าง (กู้คืนได้)

ทำไมต้องกด ถูก / ผิด

นาฬิกาไม่รู้คำตอบจริง เราจึงบอกหลังได้ผลว่านาฬิกาทายถูกหรือผิด เพื่อ (1) วัดความแม่นยำตอนใช้งานจริง และ (2) ข้อที่รู้คำตอบจะถูกบันทึกเป็นข้อมูลเทรนต่อได้ (นาฬิกาทาย "จริง" + ถูก = ตอบจริง, ทาย "จริง" + ผิด = โกหก) — กด "ไม่ทราบ" ถ้าไม่แน่ใจ ข้อนั้นจะไม่ถูกใช้เทรน

ประวัติการใช้งานจริงทั้งหมด

ทุกไฟล์ (5) ทุกเฉลย ทุกแหล่งที่มา

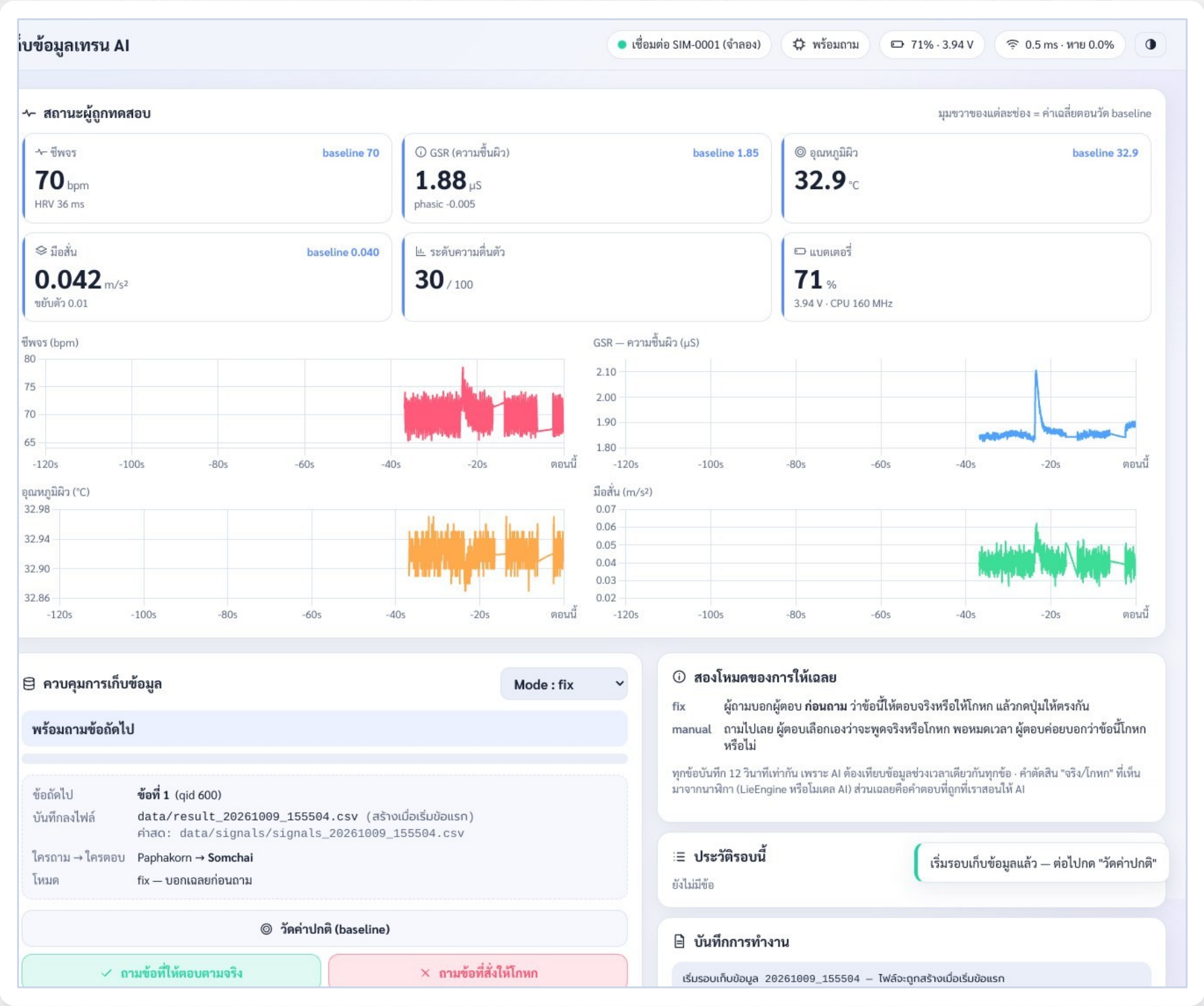
รวมทุกไฟล์เป็นไฟล์เดียว รีเฟรช

เวลา	ไฟล์ / ข้อ	qid	โหมด	ที่มา	ผู้ตอบ	คำถาม	เฉลย	นาฬิกาทาย	P(โกหก)	ตัดสินด้วย	ถูก/ผิด	ใช้เทรน	หมายเหตุ
10:00	20261009-155437	1	โหมด	ที่มา	ผู้ตอบ	คำถาม	เฉลย	นาฬิกาทาย	P(โกหก)	ตัดสินด้วย	ถูก/ผิด	ใช้เทรน	ที่มา: desktop (นาฬิกา)

- **เริ่มใช้งาน** กรอกผู้ถูกทดสอบ / ผู้ถาม → "เริ่มใช้งาน" → "วัดค่าปกติ (baseline)" ครั้งเดียวต่อคน
- **ถาม** พิมพ์คำถาม (ไม่บังคับ) → "ถาม (บันทึก 12 วินาที)" · "ยกเลิกข้อนี้" · "จบการใช้งาน"
- **หลังได้ผล** กด ถูก / ผิด / ไม่ทราบ (ถ้านาฬิกาไม่แน่ใจ: พูดจริง / โกหก) → ได้ความแม่นยำจริง + เป็นข้อมูลเทรน
- **ตัดสินด้วย** โมเดล AI / สูตรมาตรฐาน — สลับได้ทันที นาฬิกาจำไว้ใน NVS (โมเดลไม่ถูกลบ) มีค่าเตือนถ้าโมเดลไม่ดีกว่าการเดา
- **ประวัติ** กรองตามไฟล์ / เฉลย / ที่มา · ดึง "ใช้เทรน" รายข้อ · ลบข้อ (กู้คืนได้จากถังขยะ)
- **พลังงาน** หลับอัตโนมัติ · ECO 80 MHz · กำลังส่ง WiFi · พักเครื่อง · หลับลึก 60 วินาที

- นำเข้า / ส่งออก
- บันทึกเอง: data/result_<วันเวลา>.csv (mode = live)
 - ดาวน์โหลดไฟล์นี้ / รวมทุกไฟล์เป็นไฟล์เดียว (.csv)

หน้าเก็บข้อมูลเทรน AI



- **เริ่มรอบ** ผู้ถูกทดสอบ · ผู้ถาม · โหมด → "เริ่มเก็บข้อมูล" → "วัดค่าปกติ"
- **โหมด fix** กด "ถามข้อที่ให้ตอบตามจริง" หรือ "ถามข้อที่สั่งให้โกหก" ก่อนถาม = รู้เฉลยก่อน
- **โหมด manual** กดถามเลย ได้ผลแล้วผู้ตอบบอกเอง จริง / โกหก / ไม่รู้
- **หน้าจอบอก** ข้อที่เท่าไร (qid 600+) · ไฟล์ที่บันทึก · ใครถาม → ใครตอบ · นับถอยหลัง 12 วินาที · ตัวนับ จริง/โกหก/ใช้ไม่ได้/ยกเลิก
- **ปรับระหว่างรอบ** สลับ Mode fix ↔ manual · "ยกเลิกข้อนี้" · "จบรอบเก็บข้อมูล" · "ความหมายคอลัมน์ในไฟล์"
- **ทางเลือก** collect_data.bat ทำแบบเดียวกันบน command line ได้ไฟล์รูปแบบเดียวกัน

นำเข้า / ส่งออก

- data/result_<วันเวลา>.csv — 1 แถว = 1 ข้อ (เฉลย + คำตัดสิน + 12 feature)
- data/signals/signals_<วันเวลา>.csv — คำสด 5 แถว/วินาที · ไม่ได้ถามสักข้อ = ไม่สร้างไฟล์

หน้าข้อมูล & เทรน AI

ขั้นตอนเทรน AI: 1) เก็บข้อมูล (หน้า "เก็บข้อมูลเทรน AI" หรือหน้า "ใช้งานจริง" ที่บอกถูก/ผิด) → ได้ไฟล์ data/result_<เวลา>.csv → 2) ดึงไฟล์ที่จะใช้ด้านล่าง → 3) กด **เทรน AI** → 4) Studio ส่งโมเดลเข้านาฬิกาให้อัตโนมัติ (ไม่ต้องอัปโหลดเอง)

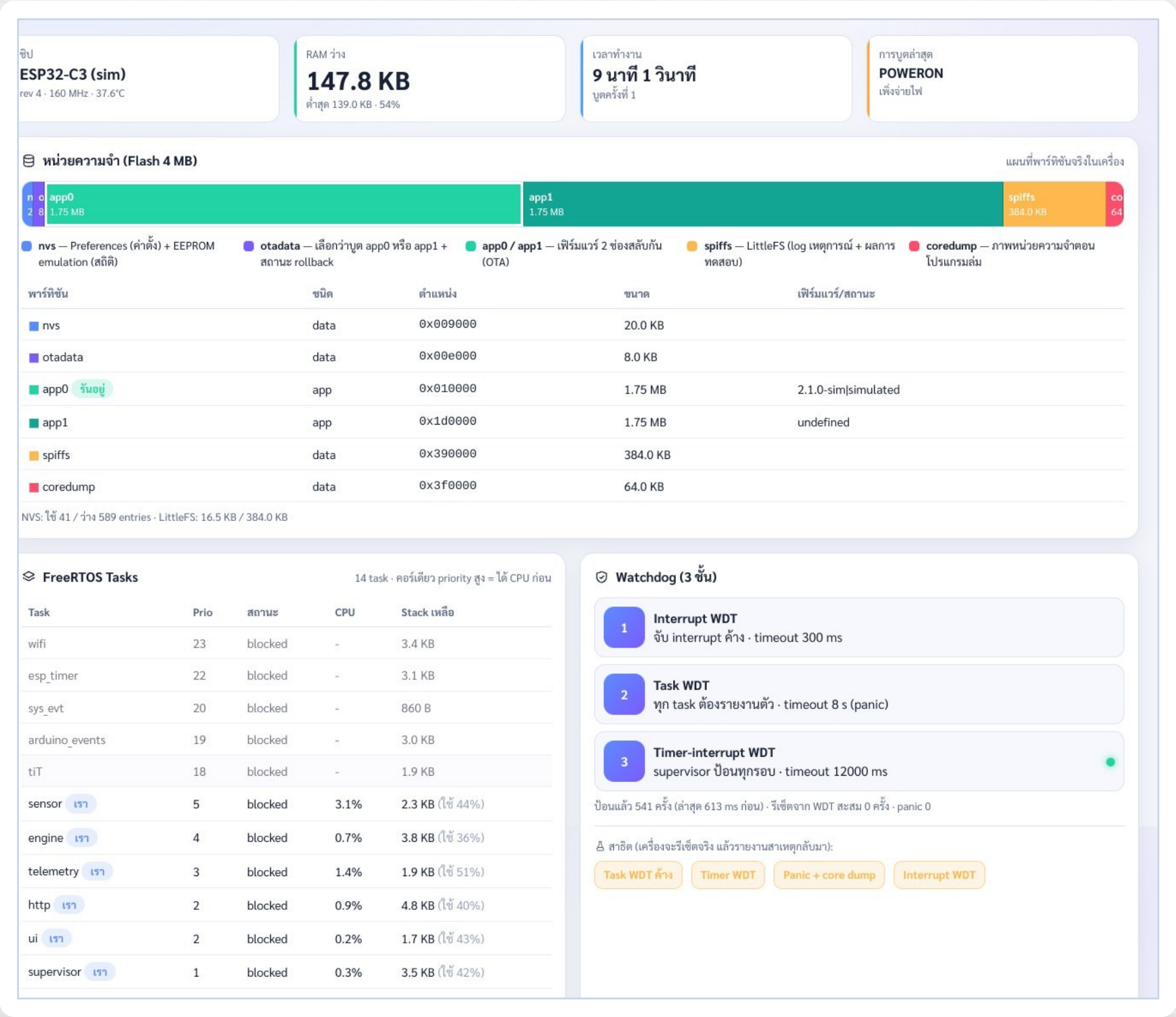
ไฟล์ข้อมูล data/result_*.csv										20 ไฟล์ - 231 ข้อ - ใช้เทรนได้ 210 ข้อ (เฉพาะไฟล์ที่ติ๊ก)		
ใช้เทรน	ไฟล์	ช่วงเวลา	ผู้ตอบ	โหมด	ที่มา	ข้อ	จริง / โทก / ไม่รู้	ใช้เทรนได้	ใช้งานจริง ถูก/ผิด			
☒	result_20261009_155437.csv	2026-10-09 15:54 → 15:54	Somchai	live	นาฬิกาจำลอง	3	2 / 1 / 0	0	3 / 0	☐	☐	☐
☒	result_20261009_131914.csv	2026-10-09 13:19 → 13:19	Tester	live	นาฬิกาจำลอง	2	1 / 1 / 0	0	1 / 0	☐	☐	☐
☒	result_20261009_121953.csv	2026-10-09 12:20 → 12:22	view	fix	คอม (Studio)	10	5 / 5 / 0	10	-	☐	☐	☐
☒	result_20261009_120902.csv	2026-10-09 12:11 → 12:15	view	manual	คอม (Studio)	16	10 / 6 / 0	16	-	☐	☐	☐
☒	result_20261008_233220.csv	2026-10-08 23:33 → 23:43	jeb	manual	คอม (Studio)	29	21 / 8 / 0	28	-	☐	☐	☐
☒	result_20261008_231630.csv	2026-10-08 23:18 → 23:29	toey	manual	คอม (Studio)	30	21 / 9 / 0	26	-	☐	☐	☐
☒	result_20261008_230717.csv	2026-10-08 23:08 → 23:10	เทส	live	คอม (Studio)	5	1 / 3 / 1	4	-	☐	☐	☐
☒	result_20261008_225913.csv	2026-10-08 23:02 → 23:03	gg	live	คอม (Studio)	2	2 / 0 / 0	2	0 / 2	☐	☐	☐
☒	result_20261008_223523.csv	2026-10-08 22:37 → 22:53	prim	manual	คอม (Studio)	32	15 / 16 / 1	30	-	☐	☐	☐
☒	result_20261008_215147.csv	2026-10-08 21:53 → 22:02	toy	manual	คอม (Studio)	30	15 / 14 / 1	29	-	☐	☐	☐
☒	result_20261008_203957.csv	2026-10-08 20:41	เทส	live	คอม (Studio)	1	1 / 0 / 0	1	0 / 1	☐	☐	☐
☒	result_20261008_202436.csv	2026-10-08 20:25 → 20:26	view	manual	คอม (Studio)	4	2 / 1 / 1	3	-	☐	☐	☐
☒	result_20261008_170335.csv	2026-10-07 20:33 → 14:01	-	watch	สำรองจากบอร์ด ESP32	7	4 / 3 / 0	7	-	☐	☐	☐
☒	result_20261008_165419.csv	2026-10-08 16:54	bank	legacy	คอม (Studio รุ่นเก่า)	1	1 / 0 / 0	0	-	☐	☐	☐
☒	result_20261008_164801.csv	2026-10-08 16:48 → 17:02	bank	manual	คอม (Studio รุ่นเก่า)	33	11 / 22 / 0	33	-	☐	☐	☐
☒	result_20261008_140045.csv	2026-10-08 14:00 → 14:01	view	fix	คอม (Studio รุ่นเก่า)	2	2 / 0 / 0	0 (ผลค้าง 2)	-	☐	☐	☐
☒	result_20261008_135934.csv	2026-10-08 13:59 → 14:00	view	fix	คอม (Studio รุ่นเก่า)	2	1 / 1 / 0	0 (ผลค้าง 2)	-	☐	☐	☐
☒	result_20261008_134705.csv	2026-10-08 13:49 → 13:57	วิว	manual	คอม (Studio รุ่นเก่า)	18	8 / 9 / 1	17	-	☐	☐	☐
☒	result_20261007_212215.csv	2026-10-07 21:22 → 21:22	-	watch	มือถือ (หน้ารับนาฬิกา)	3	2 / 1 / 0	3	-	☐	☐	☐

- **ไฟล์ข้อมูล** ดึง "ใช้เทรน" (จำใน train_exclude.txt) · ดู · ดาวน์โหลด · ลบ = ย้ายไป data/trash/
- **คอลัมน์ที่มา** คอม (Studio) · มือถือ · สำรองจากบอร์ด ESP32 · นาฬิกาจำลอง (ไม่ใช่เทรนอัตโนมัติ)
- **เทรน AI** ตั้ง "ขั้นต่ำต่อคลาส" → กดเทรน → ความแม่นยำ + ช่วงเชื่อมั่น 95% + เทียบการเดาคำตอบที่พบบ่อยสุด
- **โมเดล คอม ↔ นาฬิกา** ส่งอัตโนมัติ เปิด/ปิด · ส่ง model.json ตอนนี้ · ส่งโมเดลเก่าใน data/models/ · ลบโมเดลในนาฬิกา
- **ประวัติทุกข้อ** ดึง/เลิกใช้เทรนรายชื่อ · เลือกหลายข้อแล้วลบ (สำรองไฟล์ก่อนแก้ทุกครั้ง)
- **ถังขยะ** กู้คืน · ลบถาวร · ล้างถังขยะ

นำเข้า / ส่งออก

- **นำเข้า:** CSV จากมือถือ (polygraph_train_*.csv) · ดึงข้อมูลที่นาฬิกาบันทึกเอง — ตัดข้อซ้ำให้
- **ส่งออก:** รวมไฟล์ที่ติ๊ก / ทุกไฟล์ → result_all_<เวลา>.csv · data/model.json → นาฬิกา

หน้าระบบ & อุปกรณ์

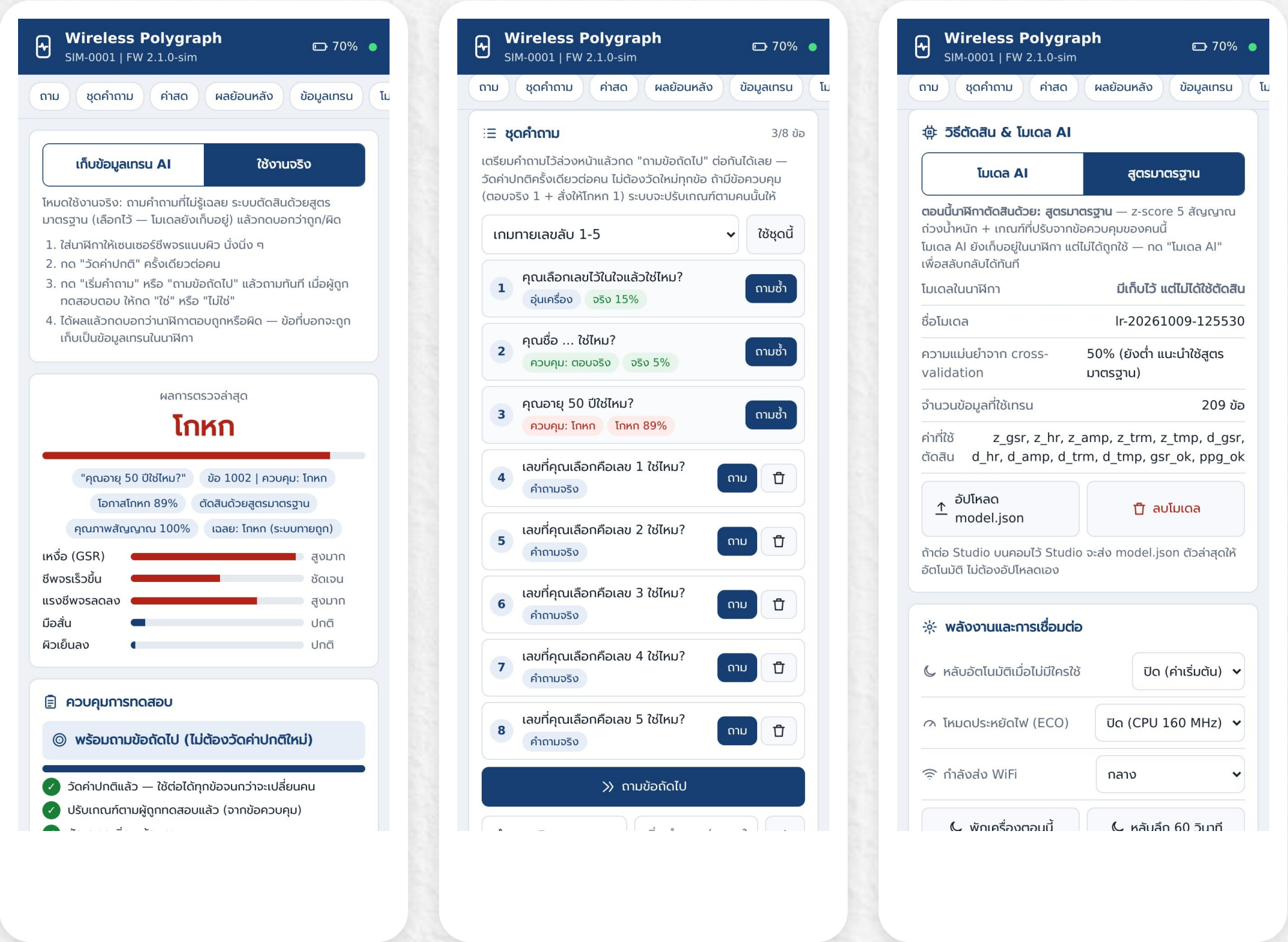


- **ดูสถานะ (ทุก 3 วิ)** ชิพ · RAM · เวลาทำงาน · สาเหตุรีบูต · Flash map · FreeRTOS task (priority, CPU %, stack) · watchdog · พลังงาน · WiFi/เซนเซอร์
- **สั่งงาน** สาธิต watchdog 4 แบบ (Task WDT, Timer WDT, Panic + core dump, Interrupt WDT) · ECO · light sleep 10 วิ · รีเซ็ต
- **ตั้งค่า LieEngine** baseline · ช่วงวัด · เกณฑ์โกหก/จริง · เกณฑ์แตะชีพจร · แรงดัน 3V3 · auto-standby → บันทึกลง NVS / คำนวณโรงงาน
- **OTA** เลือก firmware.bin → อัปเดตผ่าน WiFi (เฟิร์มแวร์ใหม่ล่ม = ย้อนเวอร์ชันเดิมเอง)
- **คู่มือ & หลักการ** สรุปวิธีใช้ หลักการตัดสินใจ ปุ่มบนนาฬิกา · ลิงก์ Data Dictionary

นำเข้า / ส่งออก

- Log ในนาฬิกา: events / results (+ ชุดก่อน) · อัปเดตทุก 5 วิ · ดาวน์โหลด .txt
- นำเข้า: firmware.bin (OTA) · ค่าตั้ง LieEngine → NVS

หน้าเว็บบนมือถือ — 192.168.4.1

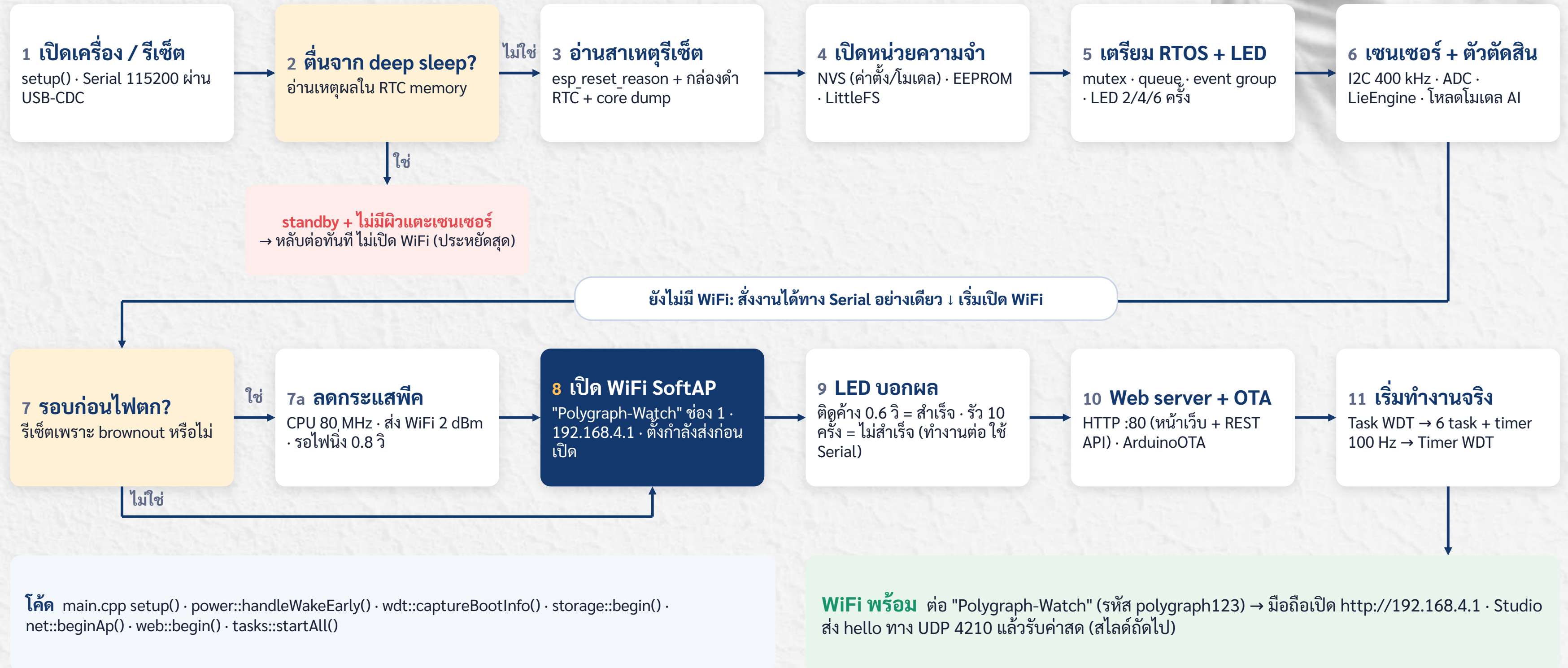


- **ไม่ต้องมีคอม/เน็ต** นาฬิกาปล่อย WiFi เอง และเป็นทั้ง backend + frontend
- **2 โหมด** เก็บข้อมูลเรสน AI / ใช้งานจริง
- **ชุดคำถาม** เลือกชุดสำเร็จรูป → "ถามข้อถัดไป" ต่อกันได้ วัดค่าปกติครั้งเดียว · เกมทายเลขลับบอกเลขที่เดา
- **หลังได้ผล** ถูก / ผิด / ไม่ทราบ → บันทึกเป็นข้อมูลเรสนในนาฬิกา + ความแม่นยำ
- **วิธีตัดสินใจ** โมเดล AI / สูตรมาตรฐาน (บอกชุดว่าตอนนี้ใช้อะไร)
- **ข้อมูลเรสน** ดูรายการ · ลบรายชื่อ · ลบข้อล่าสุด · ดาวน์โหลด polygraph_train_<วันเวลา>.csv
- **อื่น ๆ** อัปโหลด/ลบ model.json · แถบ 5 สัญญาณอธิบายผล · หลับอัตโนมัติ · ECO · กำลังส่ง WiFi

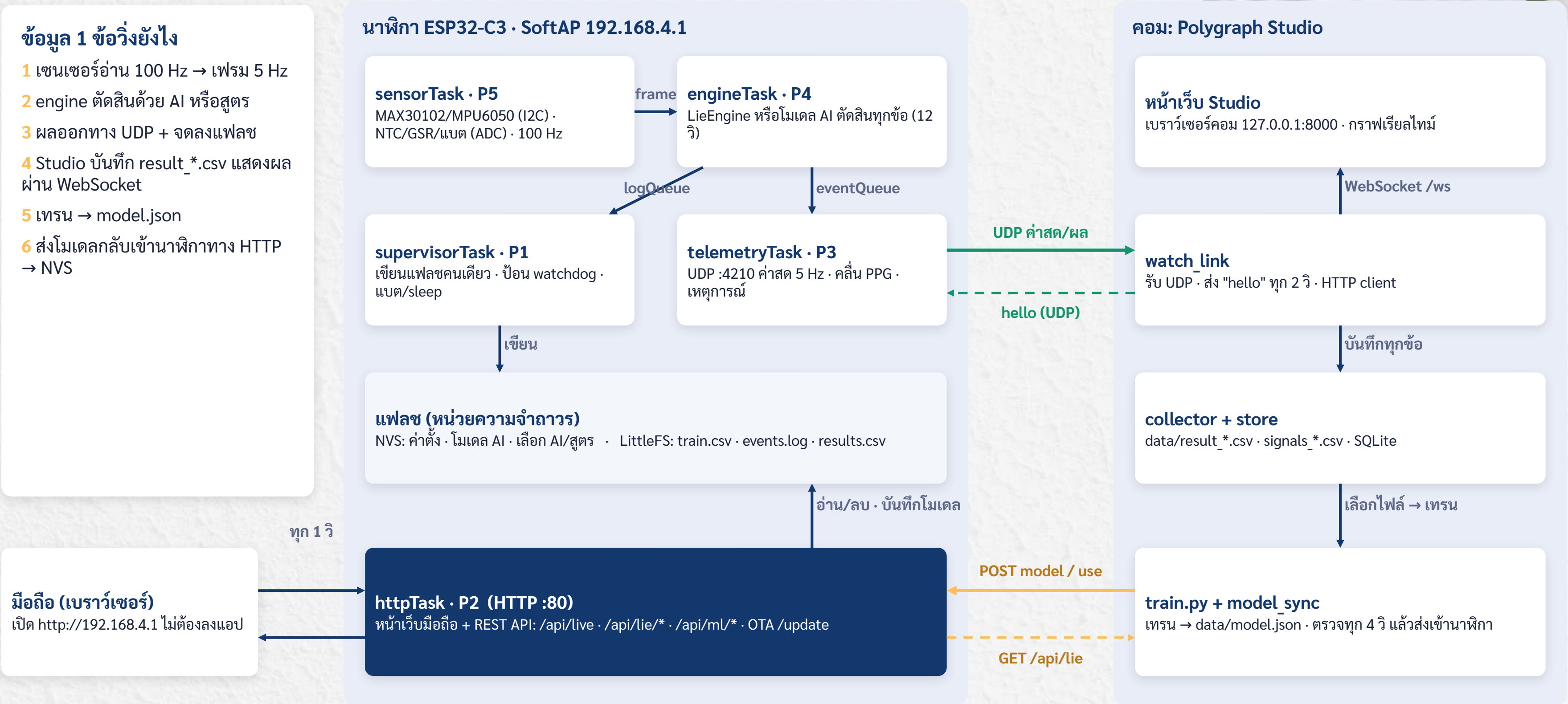
เชื่อมต่อแบบเสียบสาย vs WiFi

หัวข้อ	USB (สาย Type-C)	WiFi (SoftAP)
ใช้ทำอะไร	อัปเดตเฟิร์มแวร์ · Serial Monitor พิมพ์คำสั่ง	หน้าเว็บ · Studio · เก็บข้อมูล · OTA
ข้อมูลสด	ข้อความ/raw ทาง Serial	UDP 5 ครั้ง/วินาที + HTTP
อุปกรณ์ที่ใช้ได้	คอมเครื่องเดียว	มือถือ + คอม พร้อมกันได้ (สูงสุด 3 เครื่อง)
ความปลอดภัยตอนวัด GSR	ไม่ควรวัดกับคน (ผิวต่อกับไฟบ้านผ่านคอม)	ใช้แบตเตอรี่ แยกจากไฟบ้าน
ข้อดี	เสถียร ไม่หลุด ดีบักง่าย	อิสระ ใส่ข้อมือได้จริง
ข้อจำกัด	ติดสาย	ระยะจำกัดด้วยเสาอากาศเล็กและกำลังส่งที่ลดไว้

Flowchart: เปิดเครื่อง → ก่อนมี WiFi → WiFi พร้อม



Flowchart: WiFi ขึ้นแล้ว ข้อมูลวิ่งไปไหน



ข้อมูลเข้า-ออก: Import / Export

ข้อมูล	ไฟล์ / ปลายทาง	ทำที่ไหน (หน้า → ปุ่ม)	ทิศทาง
ผลรายชื่อ	data/result_<วันเวลา>.csv	ใช้งานจริง / เก็บข้อมูลเทรน AI → บันทึกเองทุกข้อ	สร้างเอง
ค่าสด 5 ครั้ง/วิ	data/signals/signals_<วันเวลา>.csv	ใช้งานจริง / เก็บข้อมูลเทรน AI → บันทึกเองตลอดรอบ	สร้างเอง
ไฟล์เดี่ยว	result_<วันเวลา>.csv	ตารางประวัติ → "ดาวน์โหลดไฟล์นี้" / ปุ่มลูกศรในตารางไฟล์	ส่งออก
รวมหลายไฟล์	result_all_<วันเวลา>.csv	ข้อมูล & เทรน AI → "รวมไฟล์ที่ติด" / "รวมทุกไฟล์"	ส่งออก
ข้อมูลจากมือถือ	polygraph_train_*.csv → result_*.csv	ข้อมูล & เทรน AI → "นำเข้าไฟล์ CSV (จากมือถือ)"	นำเข้า
ข้อมูลในบอร์ด	/api/ml/data.csv → result_*.csv	ข้อมูล & เทรน AI → "ดึงข้อมูลที่น่าฝึกฉบับบันทึกเอง"	นำเข้า
โมเดล AI	data/model.json (+ data/models/)	ข้อมูล & เทรน AI → "เทรน AI"	สร้างเอง
โมเดลเข้านาฬิกา	model.json → NVS ของนาฬิกา	อัตโนมัติทุก 4 วิ / "ส่ง model.json ตอนนี้" / มือถือ "อัปโหลด model.json"	ส่งออก
Log ในนาฬิกา	watch_events.txt / watch_results.txt	ระบบ & อุปกรณ์ → Log → "ดาวน์โหลด"	ส่งออก
เฟิร์มแวร์	firmware.bin → app0/app1	ระบบ & อุปกรณ์ → OTA → "อัปโหลด"	นำเข้า
ไฟล์ที่ลบ	data/trash/	ข้อมูล & เทรน AI → ถังขยะ → "กู้คืน" / "ลบถาวร"	กู้คืน

ไฟล์ทั้งหมดอยู่ใน Polygraph-Studio/data/ · ข้อจำกัดตัดตอนนำเข้า · ความหมายทุกคอลัมน์: DATA_DICTIONARY.md

05

AI & Backend

เก็บข้อมูลลง CSV · Label จริง/โกหก · เทรนและใช้บน
นาฬิกา

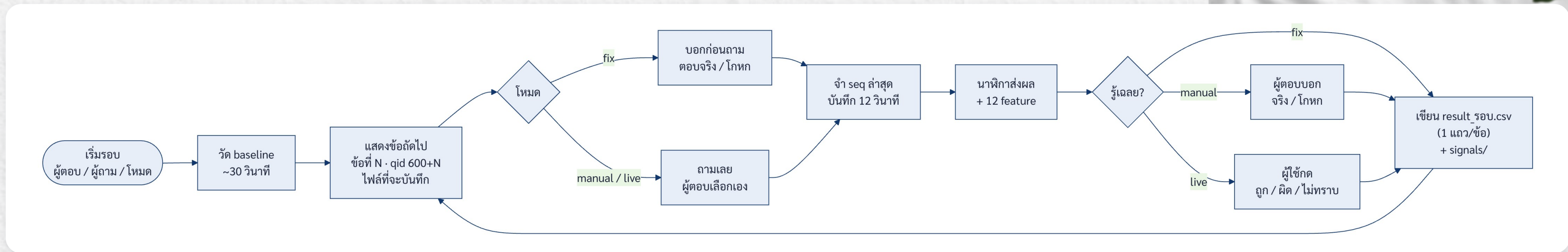


Backend: ไฟล์ .bat ทำอะไรบ้าง

ไฟล์	ทำอะไร	ใช้ตอนไหน
install.bat	ติดตั้งไลบรารี Python (FastAPI, uvicorn, websockets ...)	ครั้งแรกครั้งเดียว (ต้องมีเน็ต)
run_studio.bat	เปิด Polygraph Studio เชื่อมนาฬิกาจริง 192.168.4.1	ใช้งานจริง / นำเสนอ
run_demo_simulator.bat	เปิดนาฬิกาจำลอง + Studio + หน้าเว็บนาฬิกา	ซ้อม/ทดสอบโดยไม่มีบอร์ด
collect_data.bat	เก็บข้อมูลเทรนแบบทีละขั้น (fix/manual) ลง data/result_<วันเวลา>.csv	เก็บข้อมูลจากคอมโดยไม่เปิด Studio
train_ai.bat	เทรน AI จาก data\result_*.csv เท่านั้น → model.json (+ ส่งเข้านาฬิกาถ้าต่ออยู่)	หลังเก็บข้อมูล
train_ai_upload.bat	ส่ง model.json ที่มีอยู่เข้านาฬิกา (ไม่เทรนใหม่)	ทางเลือก — Studio ส่งให้อัตโนมัติอยู่แล้ว

โค้ด: backend/ (FastAPI) · frontend/ (HTML/JS) · ml/ (เก็บข้อมูล + เทรน, ใช้แต่ไลบรารีมาตรฐาน) · tools/virtual_watch.py (นาฬิกาจำลอง)

เก็บข้อมูลอย่างไร และทำไมเลือก CSV



เปิดด้วย Excel ได้เลย
ตรวจสอบข้อมูลด้วยตาได้ทันที

เขียนต่อท้ายทีละแถว
ไฟล์ดับ/ปิดโปรแกรม ข้อมูลก่อนหน้า
ยังอยู่

ทุกแถวบอกที่มา
question_no · watch_qid · label ·
phase

ไม่สร้างไฟล์เปล่า
สร้างเมื่อเริ่มข้อแรก

ไฟล์	1 แถว =	ใช้ทำอะไร
signals/signals_<วันเวลา>.csv	ค่าเซนเซอร์ 1 ครั้ง (5 แถว/วินาที)	กราฟ/วิเคราะห์ ว่าแต่ละข้อสัญญาณเป็นอย่างไร
result_<วันเวลา>.csv	1 ข้อ (เฉลย + คำตัดสิน + 12 feature)	ไฟล์เดียวที่ใช้เทรน AI + เทียบเฉลยกับคำตัดสิน
model.json	โมเดลล่าสุด (w, b, mean, scale)	Studio ส่งเข้านาฬิกาอัตโนมัติ

ตัวอย่างข้อมูลที่ได้ (result_<วันเวลา>.csv)

question_no	watch_qid	mode	label	verdict	p_lie	decided_by	d_hr	z_hr	z_gsr
1	600	fix	truth	truth	0.061	rules	3.99	0.15	0.24
2	601	fix	lie	lie	0.972	rules	10.64	4.58	7.08
3	602	fix	lie	lie	0.840	rules_calibrated	9.51	3.83	5.35
4	603	fix	truth	truth	0.054	rules_calibrated	3.81	0.03	-0.26

label

เฉลย: truth = ตอบจริง, lie = โกหก

verdict / p_lie

คำตัดสินและโอกาสโกหกจากนาฬิกา

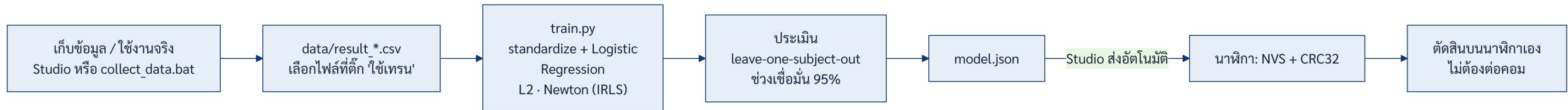
decided_by

rules = สูตร, rules_calibrated = สูตรปรับจากข้อความ, ai_model = AI

d_hr / z_hr

ชีพจรเพิ่มกี่ bpm / มากกว่าตอนพักกี่เท่า

เทรน AI อย่างไร



โมเดลและการประเมิน

Input 12 ค่า z_gsr z_hr z_amp z_trm z_tmp, d_* 5 ตัว, gsr_ok, ppg_ok

Logistic Regression + L2 $p(\text{โกหก}) = \text{sigmoid}(b + \sum w_i \cdot x_i)$ หาค่า w ด้วย Newton (IRLS)

Leave-one-subject-out เทรนด้วยคนอื่น ทดสอบกับคนใหม่ (≥ 3 คน)

รายงานผล ความแม่นยำ + ช่วงเชื่อมั่น 95%, sensitivity, specificity, เทียบกับสูตรเดิม

ml/train.py

```
# 1) อ่านเฉพาะ data/result_*.csv ที่ติดใช้เทรน
ds = build_dataset(df.training_files(DATA), feats)
# 2) ประเมินแบบไม่แอบดูข้อสอบ
method, p, _ = pm.cross_validate(ds, feats)
print(pm.metrics(ds.y, p))
# 3) เทรนจริงด้วยข้อมูลทั้งหมด
model = pm.Model.fit(ds.X, ds.y, feats, lam)
# 4) บันทึก model.json -> Studio ส่งเข้านาฬิกาเอง
save_model(model.to_json(**info))
# model_sync.py: เทียบกับนาฬิกาทุก 4 วิ แล้วส่งให้
```

AI ทำงานบนนาฬิกาเอง

```
src/lie/ml_model.cpp

float predict(const Model& m, const Result& r) {
    float x[FEAT_COUNT];
    extract(r, x);                // 12 feature
    double z = m.b;
    for (int i = 0; i < m.n; i++)
        z += m.w[i] * (x[m.idx[i]] - m.mean[i])
                / m.scale[i];
    return 1.0 / (1.0 + exp(-z)); // sigmoid
}
```

	RandomForest (โค้ดเดิม)	Logistic Regression (ที่ใช้)
รันบน ESP32-C3	ยาก (ต้นไม้ 100 ต้น)	ได้ — คูณ-บวก 12 ครั้ง
ข้อมูลหลักสืบข้อ	overfit ง่าย	เหมาะกว่า (มี L2)
อธิบายผล	ยาก	ดูน้ำหนัก w ได้ว่าสัญญาณไหนสำคัญ
เทียบความแม่นยำ	-	train.py --compare-rf

- ไม่มีโมเดล นาฬิกาใช้สูตรถ่วงน้ำหนัก (ปรับจากข้อควบคุมได้)
- มีโมเดล ทุกผลบอก "ตัดสินใจด้วยโมเดล AI" ทั้งบนเว็บ, Studio และไฟล์ CSV
- ปลอดภัย ตรวจสอบขนาด/ค่า/CRC32 ก่อนติดตั้ง ลบโมเดลได้ตลอด (model clear)

ฟีเจอร์เพิ่มเติม

1

Serial Console

พิมพ์ help · status · i2c ·
mode · data · sleep · boots ·
tasks · flash

2

OTA + Rollback

อัปเดตเฟิร์มแวร์ผ่าน WiFi ถ้า
เฟิร์มแวร์ใหม่ล้ม ย้อนกลับเอง

3

ตั้งค่า Sleep ง่าย ๆ

ค่าเริ่มต้นไม่หลับ ตั้งได้จาก
เว็บ / Serial / ปุ่ม

4

นาฬิกาจำลอง

ซ็อนนำเสนอและทดสอบ
ซอฟต์แวร์ได้โดยไม่มีบอร์ด

5

ไฟ LED 5 สี

บอกสาเหตุบูต / WiFi / แบต
ต่ำ โดยไม่ต้องมีจอ

6

ประวัติการรีเซ็ต

นับ watchdog / ไฟตก /
panic เก็บในแฟลช

7

ใช้กับโค้ดเดิมได้

ส่ง "hello" เฉย ๆ ได้ CSV รูป
แบบเดิม

8

Unit test

ทดสอบ DSP + LieEngine
บนคอม และเทียบผล C++
กับ Python

06

Limitations & Solutions

ปัญหาที่เจอ · สาเหตุ · วิธีแก้



ปัญหาที่พบและวิธีแก้

ปัญหา	สาเหตุ (ที่พบ / ที่วิเคราะห์)	วิธีแก้
ใช้แบตเตอรี่ WiFi ไม่ขึ้น / เสียบ USB แล้วเซนเซอร์ไม่ติด	สันนิษฐาน: ขา 3V3 ที่ซ็อกเก็ตต่อไม่ถึงราง 3V3 และ HT7333 จ่ายได้ 250 mA แต่ WiFi พิกัดถึง 335 mA (ตรวจด้วยมัลติมิเตอร์)	บัดกรีจุดต่อใหม่ · ตัวเก็บประจุ 470 μ F · ลดกำลังส่ง WiFi · ไฟ LED + คำสั่ง boots บอกลไฟตก
คนไม่ได้มีเหงื่อตลอด	GSR หายเมื่อผิวแห้ง ระบบเดิมพึ่ง GSR มากเกิน	ลดน้ำหนัก GSR เหลือ 20% · ตัดสินจากสัญญาณอื่นได้
Serial พิมพ์แล้วเงียบ	ตั้ง timeout การส่งเป็น 0 ทำให้ข้อความยาวถูกทิ้ง	ตั้ง 50 ms + ส่งตัวอักษรกลับ (echo)
เซนเซอร์ความเร่งเป็นรูลื่นเลียนแบบ	WHO_AM_I = 0x70 (MPU6500)	driver อ่านรูลื่นชิปแล้วตั้งค่าให้ตรง
บัส I2C ค้างเป็นบางครั้ง	สายหลวม/สัญญาณรบกวน	กู๊บัสอัตโนมัติ (ส่ง clock 9 ลูก) + watchdog

ข้อจำกัดและแนวทางต่อยอด

ข้อจำกัด

วัดความตื่นตัว ไม่ใช้การโกหกโดยตรง คนพูดจริงที่ตื่น
ตื่นอาจถูกตัดสินว่าโกหก

โกหกตามคำสั่ง $\neq$ โกหกจริง ความแม่นยำใช้ได้กับ
สถานการณ์ทดลองเท่านั้น

ข้อมูลยังน้อย ต้องเก็บหลายคน รายงานช่วงเชื่อมั่นทุก
ครั้ง

ESP32-C3 แขนเดียว **RAM 400 KB** ต้องใช้โมเดลเบา
ระยะ **WiFi** เสืออากาศเล็ก + ลดกำลังส่งเพื่อลดกระแส

แนวทางต่อยอด

- เปลี่ยน LDO เป็นรุ่นจ่าย ≥ 500 mA เช่น AP2112K-3.3
- เก็บข้อมูลหลายคนขึ้น แล้วเทียบโมเดลแบบอื่น
- เพิ่ม Bluetooth LE สำหรับมือถือโดยไม่ต้องสลับ WiFi
- ทำกล่อง/สายรัดที่กดเซนเซอร์แบบผิวสัมผัส
- ใช้เพื่อการศึกษาเท่านั้น ไม่ใช่ตัดสินคนจริง

DEMO

- 1 ใส่नाฬิกา ดูค่าสดและกราฟบนจอ
- 2 วัด baseline 30 วินาที
- 3 ถ้ามั่วข้อมูล: ตอบจริง / สั่งให้โกหก
- 4 ถ้ามั่วถามจริง → ผลจริง/โกหก + เหตุผล
- 5 ลองให้ผ่าน GSR ไม่แตะผิว → ยังตัดสินใจได้
- 6 สานิต watchdog → รีเซ็ตแล้วบอกสาเหตุ





Thank you

ขอบคุณครับ / ค่ะ — ยินดีตอบคำถาม

อัจฉรา ตั้งดี 6730300655 · ปภากร จันทร์ดี 6730300809

03603323 Introduction to Embedded Systems · วิศวกรรมคอมพิวเตอร์ มก. ศรีราชา